

Towards Unified and Data-Efficient Prognostics and Health Management with Tabular Foundation Models

Raffael Theiler^{a,*} (Researcher), Lev Telyatnikov^{a,*}, Leandro Von Krannichfeldt^a and Olga Fink^a

^aIMOS Lab, EPFL, Lausanne, Switzerland

ARTICLE INFO

Keywords:

Tabular Foundation Models
Prognostics
Time Series
Transformer

ABSTRACT

Data-driven Prognostics and Health Management (PHM) uses time-varying condition-monitoring data to diagnose system states and estimate remaining useful life in engineered assets. These tasks are central to maintenance planning, but industrial PHM data are often fragmented, partially observed, and poorly labeled, which hinders supervised learning. Foundation models offer a route toward reusable predictive systems, yet most time-series foundation models are designed for forecasting and assume long, coherent, regularly sampled sequences. To address this gap, we propose a framework for applying Tabular Foundation Models to industrial time series using in-context learning, and we evaluate them on a variety of PHM tasks. By converting raw unit-level signals into tabular rows, we show that these models perform well across multiple tasks - including prognostics, and diagnostics — and are highly data efficient. We compare them directly with sequence models, transformer baselines, and gradient-boosted trees under a common evaluation protocol. The results indicate that tabular foundation models achieve the best average ranks across prognostic and diagnostic tasks. Our findings further show that PFN-based models are competitive in low-data regimes, that temporal context can be preserved in the tabular representation, and that performance depends on representative context construction under subsampling. These results demonstrate that tabular foundation models provide a practical and general interface for heterogeneous PHM problems.

1. Introduction

Prognostics and Health Management (PHM) has matured into an established framework for supporting maintenance and operational decisions in complex engineering systems. In industrial practice, PHM solutions are typically based on condition monitoring data collected from heterogeneous sensor networks and are used to support fault detection, diagnostics, and, where feasible, remaining useful life (RUL) prediction [67, 34]. These outputs are then integrated into maintenance planning and asset management processes to reduce downtime and maintenance costs while maintaining high system availability.

A cornerstone of PHM is the availability of degradation trajectories and time-to-failure information, as these data capture the temporal evolution of system health and form the basis for many prognostic approaches [34]. Despite their importance, such trajectories are rarely available in operational environments. Failures are relatively rare events, assets are often maintained or replaced before reaching end of life, and historical data may be fragmented or incomplete. As a consequence, many prognostic approaches developed and evaluated in academic settings cannot be readily deployed in practice [19]. Instead, most industrial PHM implementations today primarily focus on fault detection, where data from normal operation are abundant, and the objective is to identify deviations from healthy behavior rather than to predict precise failure times.

Recent advances in machine learning, particularly deep learning, have aimed to reduce reliance on handcrafted features by learning representations directly from data [32, 65, 55]. While such approaches have demonstrated strong performance in controlled settings, their success often relies on assumptions that are difficult to satisfy in industrial environments. In particular, training deep models from scratch typically requires large amounts of labeled data that cover the full range of operating conditions and fault types. In industrial PHM, however, labeled failure data are scarce, system configurations evolve over time, and degradation trajectories are rarely observed until the end of life [35]. As a result, models trained under one set of conditions often require costly retraining, adaptation, and validation before they can be deployed across fleets of assets.

*Equal contribution and Corresponding author

 raffael.theiler@epfl.ch, lev.telyatnikov@epfl.ch (R. Theiler)

ORCID(s): 0000-0000-0000-0000 (R. Theiler)

In current industrial PHM practice, models are often developed and trained for individual components or specific systems, reflecting differences in design, sensing setups, and operating conditions [24, 40, 63]. However, in real deployments these models must operate across fleets of assets and multi-component systems within the same organization. Assets frequently differ in age, configuration, and operating regimes, and their behavior may evolve over time due to changes in usage patterns, maintenance strategies, or environmental conditions [19]. Continuously retraining and revalidating separate models for each component, asset, or configuration is costly and difficult to scale, particularly in safety-critical applications. This lack of scalability remains one of the key barriers to broader adoption of advanced PHM methods in industry. Learning approaches that can generalize across assets and adapt to new conditions without repeated retraining are therefore of high practical relevance.

Under these constraints, in-context learning provides an alternative to training models from scratch [13, 36]. Rather than updating model parameters, foundation models adapt their behavior at inference time by conditioning on a small set of task-specific examples [66]. This mechanism is particularly well suited to PHM applications, where labeled data are limited, system behavior varies across assets, and rapid adaptation to changing operating conditions is required. By enabling task adaptation without retraining or parameter updates, in-context learning reduces deployment and maintenance overhead while remaining compatible with established PHM workflows.

While time-series foundation models have recently been proposed and extended to in-context learning [1], their applicability in industrial PHM is constrained by both data assumptions and task formulation [41, 56]. These models are typically designed for forecasting tasks, where the objective is to predict future signal values over a temporal horizon [59, 12, 37], and effective adaptation relies on long, coherent temporal sequences with stable dynamics, consistent sensor availability, and well-aligned sampling across channels. In contrast, prognostics problems are typically framed as regression tasks, such as estimating health indicators or remaining useful life from the current system state, often without access to complete or continuous degradation trajectories. Moreover, in industrial PHM settings, condition monitoring data are frequently irregular, affected by missing values, and disrupted by maintenance actions or changes in operating conditions. These factors limit the suitability of time-series foundation models, particularly for prognostics applications.

At the same time, the absence of continuous run-to-failure trajectories motivates learning paradigms that operate on snapshots of system state rather than complete time-to-failure trajectories. In practice, PHM models often rely on tabular representations composed of aggregated sensor features, condition indicators, and contextual variables [50, 65]. Such representations allow information to be pooled across time and assets, are robust to interruptions caused by maintenance actions, and do not require strict temporal alignment between sensors. This is particularly important in industrial environments, where sensor signals often exhibit different sampling rates, heterogeneous temporal dynamics, and varying levels of reliability, and where data streams are frequently affected by missing values due to sensor faults, communication issues, or changes in instrumentation [24, 40, 63]. Handling these characteristics in sequential models typically requires additional assumptions, resampling, or imputation strategies, whereas tabular representations and models can more naturally accommodate incomplete, irregular, and heterogeneous observations.

Within this setting, tabular foundation models combined with in-context learning provide a strong alternative to conventional end-to-end training paradigms [26, 27, 38]. By leveraging a single pre-trained model across tasks and assets, these approaches reduce the need for repeated architecture-specific hyperparameter optimization; the remaining tuning is concentrated mainly on tabular-shape choices such as lookback length, feature construction, and context size. Adaptation to evolving operating conditions, changing sensor availability, or new assets can be achieved through contextual examples at inference time, rather than through parameter updates, thereby avoiding costly retraining cycles and simplifying deployment and lifecycle management.

2. Related Work

2.1. Time-series Foundation Models

Recently, the broader field of machine learning has witnessed the emergence of *Foundation Models* (FMs) as powerful, general-purpose architectures, leading to a profound impact across multiple disciplines in science [66]. FMs are large neural networks pre-trained on diverse and heterogeneous datasets; the unprecedented scale of this pretraining equips them with emergent general-purpose capabilities, allowing them to learn general, transferable latent representations and priors that can be reused across tasks and datasets. Especially for Natural Language Processing, this often allows FMs to outperform domain-specialized models with minimal task-specific adaptation [43].

There has been substantial interest in reproducing this success in the field of time-series analysis, generalist Time-Series Foundation models. Inference for these models may be seen as generally analogous to that of the large language models: continuous-time measurements are sampled, then embedded as discrete high-dimensional tokens, which are then fed to a deep learning model composed of self/cross-attention layers and multilayer perceptrons. Time-series Foundation Models (TSFMs) extend Time-series Transformers architecture beyond task-specific design, to enable zero-shot or few-shot generalization across datasets and domains. In general, the research of TSMs can be divided into three categories [41]. One category concerns itself with the repurposing of language models for time series tasks, such as Time-LLM [30]. Hereby, time series sequence is converted into a word structure, divided into patches, and used as input tokens for the language foundation model. The second category leverages a pre-train on a large variety of data. The different works either train exclusively on real-world data [59], only on synthetic data [14] or a combination of real measurements and synthetically generated samples [12, 2]. The third category creates domain-specific TSFMs by training directly on domain-relevant datasets like BioBERT for biomedical literature [33] or PowerPM for Power System tasks [54]. Their idea is to improve performance for specific domain tasks, albeit incurring a reduced generalization ability.

The majority of TSFM works focus on univariate forecasting by treating each time series independently [17], and consequently do not make effective use of covariance dependencies of a multivariate time series input. Some publications try to mitigate this effect by constructing a new time series out of the multivariate series through a flattening operation [59] or subsampling [37]. Recent works recognize the potential of in-context learning for covariate information integration, and are slowly shifting towards in-context learning for TSFM's [1].

2.2. Tabular Foundation Models

Tabular Foundation Models (TFMs) have recently emerged as transformer-based FMs for two-dimensional data, including tables and relational databases. TFMs are trained to predict cells in tables by leveraging contextual information from adjacent rows and columns, allowing them to fill missing values, extrapolate rows and columns, and detect anomalies. This is achieved by training on vast amounts of tabular data, coupled with a two-dimensional attention mechanism that captures both row-level and column-level dependencies. These capabilities effectively make TFMs universal pattern recognizers for table-like data, and have achieved state-of-the-art performance on multiple tabular prediction tasks.

TFMs differ on how their attention mechanisms are implemented, as well as on the nature of their training data, leading to multiple implementations in the literature. *Tabular Prior-Fitted Network* (TabPFN) [26, 27], emulates Bayesian inference on tables by applying attention to raw table data, and is trained on massive causally-generated synthetic datasets; *TabDPT* combines synthetic and real datasets to improve generalization and one-shot capacity [38]; *TabICL* uses statistical row embeddings to compress tables, enabling scalable extensions to hundreds of thousands of samples without fine-tuning [45]; *Carte* embeds tables with graph-like structure to extract semantic connections between feature columns [31].

The success of TFMs has led to these models being used for a variety of two-dimensional signals, including multivariate time series. This typically takes place by representing time series as tables and embedding time markings as additional feature columns. [28] demonstrate on small-scale, scalar time series that a simple tabularization and feature-based representation of time series allows TabPFN to match or outperform specialized forecasting models, highlighting the strength of large-scale synthetic pretraining. Similarly, [7] further adapt TabPFN for time-series forecasting by explicitly encoding time-periodic features as sinusoidal table features, showing consistent improvements over baseline TabPFN and other time series forecasting models for small quasi-periodic time-series.

While the use of tabular foundation models for time-series forecasting has recently gained attention, it remains unclear how to fully exploit the time dependencies within and across time series beyond simple periodic indexing or feature lagging, especially under the computational constraints of tabular foundation models.

2.3. Foundation Models for PHM

The success of FMs for sequence modeling has led to substantial interest in using them for industrial time series across multiple applications. Despite numerous FM architectures being proposed to enhance state-of-the-art performance in various time-series-related tasks [56], all-encompassing FM for PHM have seen limited adoption in industry, where classical models such as Particle Filters, LSTM and CNN are still the most widely adopted [47]. Recent studies in the PHM literature have begun to utilize TSFMs. An example is UniFault [16], which proposes large-scale pretraining for rotating machinery fault diagnosis, with the goal of improving cross-domain transfer and few-shot

adaptation when only limited labeled data are available for a new machine or condition. Another example is Yao and Han [62], where the streamlining of prognostics and maintenance of wind turbines with generalization capabilities of TSFMs is conceptually outlined. The strong performance of TFMs has also led to exploratory applications in PHM. Magadán et al. [39] use TabPFN for early fault classification in rotating machinery, showing that it maintains strong diagnostic accuracy even with very limited training samples. Sun et al. [51] propose a hybrid TimeGAN–TabPFN framework for slewing bearing fault diagnosis using audible sound signals, where generative data augmentation combined with TabPFN significantly improves classification performance under small-sample conditions.

Although tabular foundation models are slowly gaining traction in the PHM field with domain-specific pretraining and several diagnostics application studies, a systematic benchmarking study with comprehensive evaluations in both prognostics and diagnostics is still missing.

2.4. Benchmarks in PHM

In recent years, several efforts have been conducted in terms of systematic framework proposals and benchmarking evaluations in PHM. An early review study was presented by Ramasso and Saxena [46], who analyzes prognostic methods predating modern deep-learning, developed using the C-MAPSS turbofan degradation datasets [48]. The study examines how the C-MAPSS dataset has been used across prior work, clarified differences among the dataset variants, and reconstructed benchmark results from the PHM 2008 Data Challenge. It also highlights inconsistencies in dataset selection, preprocessing, and metric usage, and provides guidelines for more consistent benchmarking. More recently, Qiao et al. [44] carried out a comparative benchmarking evaluation of deep-learning models for bearing fault diagnosis and fault prognosis using the XJTU-SY Yaguo et al. [61] and CWRU [8] datasets. The study evaluated CNN, LSTM, ResNet, Transformer, and hybrid architectures, identifying ResCNN-LSTM as a strong model for both diagnosis and prognosis. However, its scope is limited to conventional deep-learning architectures and does not consider TSFMs or TFMs. PDMBench [63] proposes a standardized platform for predictive maintenance research that integrates 14 public datasets covering bearings, motors, gearboxes, and multi-component systems, together with 22 baseline models ranging from traditional machine-learning methods to modern Transformer-based architectures. The framework addresses practical challenges such as irregular sampling rates, imbalanced fault distributions, heterogeneous sensor modalities, and deployment constraints, while focusing on fault classification and remaining useful life prediction. Its experimental analysis shows that Transformer architectures perform strongly on structured bearing datasets but are less robust when applied to noisy motor signals, whereas lightweight models provide attractive accuracy–efficiency trade-offs for deployment. Although PDMBench provides a systematic benchmarking framework, it does not standardize evaluation protocol as well as task semantics and does not evaluate foundation-model approaches. Beyond dataset-level standardization, recent work has also addressed the reproducible implementation of conventional PHM methods. Telyatnikov et al. [52] introduce PICID as a modular PHM evaluation framework for specifying datasets, preprocessing, target construction, model interfaces, and metrics under a shared executable protocol. This framework has been used to implement and evaluate conventional PHM papers as reproducible benchmark components [53].

Several conceptual as well as experimental benchmarking publications have been carried out with a strong focus on standard deep-learning models and Transformer architectures. Therefore, a rigorous and reproducible investigation of tabular foundation models for prognostics and diagnostics has not yet been addressed.

In this work, we use PICID as the evaluation infrastructure for assessing tabular foundation models under the same task, dataset, preprocessing, and metric interfaces as their conventional PHM competitors. In particular, PICID’s fit–predict model interface enables tabular foundation models to be benchmarked against conventional baselines through the same executable protocol, making conventional PHM methods available as standardized competitors rather than as isolated, paper-specific implementations.

3. Methodology

In this work, we formalize a unified methodology for transforming raw PHM time-series into representations compatible with both sequence-based models and tabular foundation models. The central objective is to ensure that all models are evaluated on information-equivalent inputs, thereby isolating the impact of the modeling architecture from differences in preprocessing.

A key challenge arises from the mismatch between the inherently sequential nature of PHM data and the requirements of tabular foundation models, which operate on collections of independent samples and, in the case of in-context learning, rely on well-defined context sets. To bridge this gap, we define a data processing pipeline that

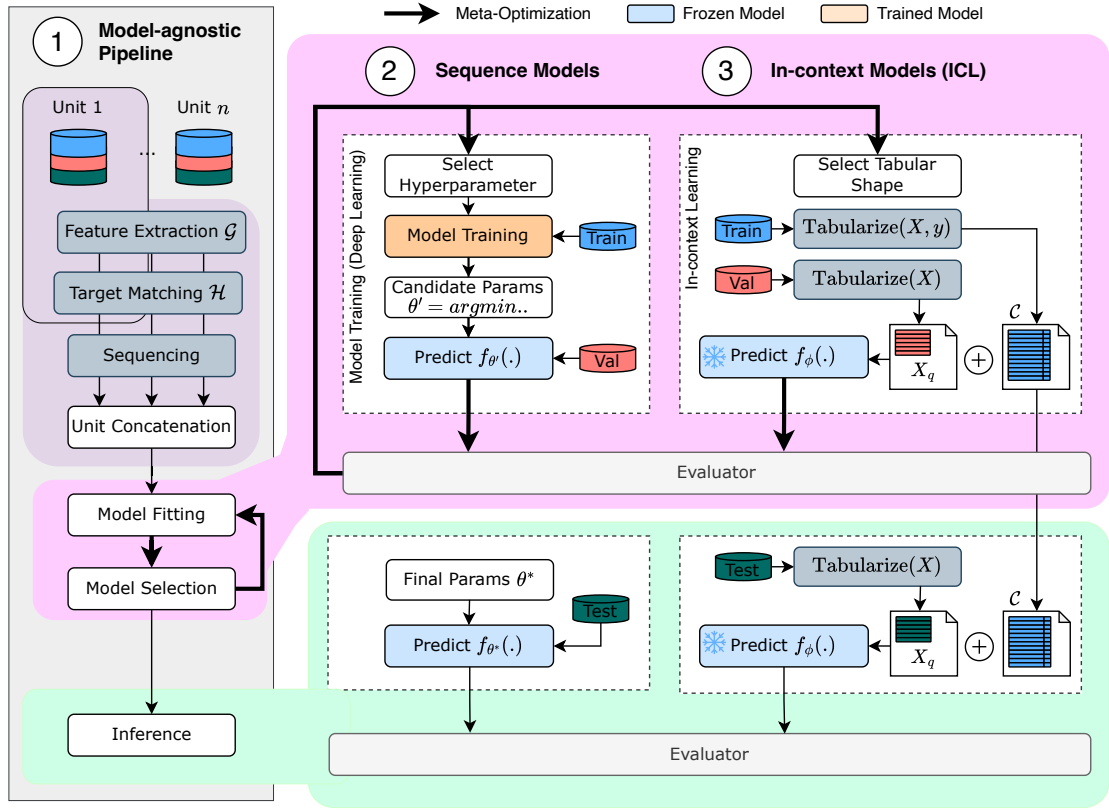


Figure 1: Overview of the unified evaluation pipeline. Unit-level PHM signals are transformed into aligned feature–target windows and evaluated either by trained sequence models or by in-context tabular foundation models after tabularization. Validation data are used for model selection or tabular-shape selection before final test evaluation.

converts continuous time-series into supervised samples that can be interpreted consistently across model families. To this end, we introduce a complete data processing pipeline that spans feature extraction, target alignment, sequence construction, and tabularization. This pipeline allows us to compare fundamentally different model families under controlled and consistent conditions.

Figure 1 provides a high-level overview of the proposed framework. This section is structured as a step-by-step pipeline. We first define how raw signals are transformed into aligned feature–target sequences. We then describe how supervised samples are constructed through sequence slicing, and finally how these samples are converted into tabular representations via the tabularization operator.

3.1. Data Pre-Processing Pipeline

We formalize tabularization as a multi-stage transformation pipeline that maps raw time-series data into supervised learning representations. Specifically, the pipeline consists of three stages: (i) transforming raw signals into a compact feature representation ($\mathcal{G}$), (ii) slicing the transformed features into temporal sequences ($\mathcal{S}$), and (iii) mapping these sequences into tabular representations via the tabularization operator ($\mathcal{T}$). A key aspect of this formulation is that target alignment is defined explicitly. This allows us to accommodate feature transformations that modify the temporal grid (e.g., time-frequency representations), while remaining agnostic to the specific alignment convention.

Importantly, this explicit formulation ensures that the resulting samples are:

- well-defined as supervised learning instances, and
- directly usable as context elements for tabular foundation models.

Table 1
Summary of Notation

Symbol	Description
<i>Raw Data Space</i>	
T	Total duration (number of timestamps) of the raw unit lifecycle
t	Discrete raw time index, $t \in \{1, \dots, T\}$
$\mathbf{x}(t)$	Raw sensor vector at time t , $\mathbf{x}(t) \in \mathbb{R}^M$
$y(t)$	Raw target value at time t
<i>Data Transformation ($\mathcal{G}, \mathcal{H}$)</i>	
T'	Total number of steps in the transformed time-series
w	History window size (receptive field) for feature extraction
$s_{\mathcal{G}}$	Signal processing stride (step size for sliding the transformation window)
Ψ, Φ	Collections of fitted parameters for feature and target transformations
$\mathcal{G}(\cdot; \Psi)$	Feature transformation operator for sensor signals
$\mathbf{z}(j)$	Transformed feature vector at feature-step j , $\mathbf{z}(j) \in \mathbb{R}^F$
$z_y(j)$	Transformed target value at feature-step j
$a(j)$	Raw-time support associated with feature-step j (timestamp or interval)
$\mathcal{A}(\cdot)$	Alignment/aggregation operator used within the target pipeline
$\mathcal{H}(\cdot; \Phi)$	Target transformation operator (e.g., RUL normalization or scaling)
$\tilde{\mathcal{H}}(\cdot; \Phi)$	Target pipeline that produces one label per transformed index using $a(j)$
<i>Dataset Slicing ($\mathcal{S}$)</i>	
L_{seq}	History length (number of transformed steps per input window)
Δ	Stride between consecutive window starts in transformed time
ρ	Warm-start depth (left-padding allowance), $\rho = 0$ strict windowing
δ	Supervision offset (steps after window end), $\delta = 0$ end-of-window
L_{pred}	Supervision segment length (default $L_{\text{pred}} = 1$ for PHM window labels)
$\mathbf{W}_m$	Temporal feature window
N_{slices}	Total number of windows extracted from the unit
$\mathcal{K}$	Admissible set of window start indices in transformed time
<i>Tabularization ($\mathcal{T}$)</i>	
$\mathbf{X}_m$	Final tabular input vector for the model
$\mathcal{D}_{\text{seq}}$	Dataset in sequence format (for time-series models)
$\mathcal{D}_{\text{tab}}$	Dataset in tabular format (for tabular models)
D_{tab}	Tabular dimensionality, $D_{\text{tab}} = L_{\text{seq}} F$
<i>Modeling & Evaluation</i>	
$f_{\theta}(\cdot)$	Deep Learning Model (Sequence-based)
$f_{\phi}(\cdot)$	Foundation Model (Tabular/In-Context)
$\mathcal{C}$	Context set for In-Context Learning
$\mathcal{L}$	Loss function

Table 1 summarizes the notation used throughout this pipeline.

3.1.1. Problem Formulation: Single Unit Perspective

We begin by formalizing the data representation for a single functional unit monitored continuously over its full operational lifecycle. This setting serves as the fundamental building block for the subsequent pipeline, which

transforms raw time-series into supervised learning samples. The raw data consists of a multivariate time-series of sensor measurements $\mathcal{X}$ and a corresponding target signal $\mathcal{Y}$:

$$\mathcal{X} = \{\mathbf{x}(t)\}_{t=1}^T, \quad \mathcal{Y} = \{y(t)\}_{t=1}^T \quad (1)$$

where T denotes the total number of recorded timestamps. At each time step t , the system state is represented by a vector of sensor observations $\mathbf{x}(t) \in \mathbb{R}^M$, defined as :

$$\mathbf{x}(t) = [x_1(t), x_2(t), \dots, x_M(t)]^\top \quad (2)$$

where $x_m(t)$ represents the scalar measurement of the m -th sensor at time t . These channels may correspond to endogenous system variables or exogenous covariates. The target signal $y(t)$ encodes the system health state at time t . Depending on the PHM task, this may represent a continuous quantity (e.g., remaining useful life for prognostics) or a discrete label (e.g., fault type for diagnostics). This formulation defines the raw data space on which all subsequent transformations operate. In particular, it establishes the relationship between sensor observations and the target signal at the original temporal resolution, which will serve as the reference for all downstream processing steps.

Example. Consider a lithium-ion battery cell from the UNIBO21 dataset [57] monitored over its operational lifetime. During each discharge cycle, three signals are recorded at regular intervals: voltage, discharge current, and cell temperature, so $M = 3$ in this example. The target $y(t)$ is the remaining cumulative discharge throughput, or ah-RUL, which decreases from the first recorded cycle toward zero at end-of-life. This raw pair $(\mathcal{X}, \mathcal{Y})$ serves as input to the pipeline.

The goal of this stage is to transform the raw time-series into a feature representation that can be used to construct supervised learning samples while ensuring consistency between inputs and targets. In PHM, preprocessing often constitutes a substantial part of the methodological pipeline. It includes operations such as data cleaning, normalization, resampling, windowing, detrending, denoising, and signal-processing transforms. Although these stages typically receive less emphasis than the predictive model, they are at least equally important for final performance and reproducibility. A central challenge is that feature extraction procedures may modify the temporal structure of the data, while the target signal remains defined on the original timeline. As a result, we must explicitly define how features and targets are brought into correspondence.

3.1.2. Feature and Target Transformation Operators

The first stage pre-processes the raw time-series into a feature representation that can be used to construct supervised learning samples while ensuring consistency between inputs and targets. In PHM, preprocessing is a critical component of the pipeline, often encompassing data cleaning, normalization, resampling, windowing, detrending, denoising, and signal-processing transformations. Despite receiving less attention than the predictive model, these steps are equally important for performance and reproducibility. To address feature extraction and target processing, we decompose the transformation into two operators:

- a feature transformation $\mathcal{G}$
- a target transformation pipeline $\mathcal{H}$

Target alignment is treated explicitly as part of the pipeline rather than being implicitly induced by the feature representation. This design decouples feature extraction from supervision design, thereby supporting arbitrary feature transformations, including those that alter temporal resolution, while preserving a consistent definition of the target under different alignment conventions.

We define the feature transformation $\mathcal{G}$ as a composition of P processing functions:

$$\mathcal{G}(\cdot; \Psi) = g_P(\cdot; \psi_P) \circ \dots \circ g_1(\cdot; \psi_1) \quad (3)$$

where Ψ denotes the set of fitted parameters (e.g., normalization statistics). Applying the feature transformation $\mathcal{G}$ to the raw sensor series yields a transformed feature series:

$$\mathcal{Z} = \{\mathbf{z}(j)\}_{j=1}^{T'} = \mathcal{G}(\mathcal{X}; \Psi), \quad \mathbf{z}(j) \in \mathbb{R}^F \quad (j = 1, \dots, T'). \quad (4)$$

Similarly, we define the target transformation operator $\mathcal{H}$ as a composition of Q distinct processing functions:

$$\mathcal{H}(\cdot; \Phi) = h_Q(\cdot; \phi_Q) \circ \dots \circ h_1(\cdot; \phi_1) \quad (5)$$

where Φ denotes the set of fitted parameters used for target preprocessing, estimated exclusively from the training partition (e.g., selection, scaling or clipping parameters). Importantly, $\mathcal{H}$ operates on the target signal itself and does not perform temporal alignment. Its role is to transform the target values into a suitable representation for learning.

We have now introduced the feature and target transformation operators. However, many commonly used feature transformations, such as time–frequency representations (e.g., STFT or wavelets), operate on temporal windows and therefore produce one feature vector per window rather than per original timestamp. This leads to a mismatch between the feature timeline and the raw target timeline. Without a precise alignment strategy, the definition of supervision becomes ambiguous. In addition, it is often necessary to pre-process the target signal before it is used for supervision. For example, in prognostics tasks, the raw target (e.g., remaining useful life) may be clipped, normalized, or rescaled to improve numerical stability and learning behavior. Together, these considerations necessitate an explicit alignment mechanism.

Temporal alignment We explicitly define how features and targets are brought into correspondence. To this end, each transformed index j is associated with a raw-time support $a(j) \subseteq \{1, \dots, T\}$, which specifies the portion of the original time axis from which the feature $\mathbf{z}(j)$ is derived.

The support $a(j)$ may take one of two forms: it can be a single timestamp $a(j) \in \{1, \dots, T\}$ (e.g., the end or center of a window), or an interval $a(j) = [\bar{t}_j, \bar{t}_j]$ (e.g., the window used to compute the feature).

To map this support to a supervision signal, we introduce an alignment operator $\mathcal{A}$ that extracts a representative target value from the raw target trajectory $\mathcal{Y}$ over the support $a(j)$. This yields an aligned target value at index j :

$$z_y(j) = \mathcal{A}(\mathcal{Y}, a(j)). \quad (6)$$

Depending on the form of $a(j)$, $\mathcal{A}$ may correspond to pointwise sampling (for timestamp supports) or aggregation over a window (e.g., last-value, mean, or max for interval supports). This alignment step is used solely to define supervision and is not provided as input to the model.

Feature-to-raw-time mapping via temporal alignment and temporal indexing. When $\mathcal{G}$ modifies the temporal resolution, each transformed index j must be related back to the original time axis. Using the temporal alignment defined above, we associate each feature vector $\mathbf{z}(j)$ with its corresponding raw-time support $a(j) \subseteq \{1, \dots, T\}$, which specifies the portion of the original time-series from which the feature was derived.

The resulting feature dimension F , as seen by the model after applying $\mathcal{G}$, is determined by the final stage operator g_p (e.g., $F = M \times \text{features per channel}$). Each stage g_p operate either pointwise in time (e.g., scaling), or over temporal windows (e.g., time–frequency transformations such as STFT or wavelets). Consequently, the transformation stage may alter both the feature dimension and the temporal resolution.

In many practical implementations, the index set $\{1, \dots, T'\}$ of the transformed sequence $\mathcal{Z}$ is induced by one or more windowed stages within the pipeline (e.g., STFT or wavelets), which operate with a history length w and stride s_G , thereby producing one feature vector per window, yielding

$$T' = \left\lfloor \frac{T - w}{s_G} \right\rfloor + 1. \quad (7)$$

For example, if the windowed stage uses window endpoints $\bar{t}_j = w + (j - 1)s_G$, then the raw-time support can be written as $a(j) = [\bar{t}_j - w + 1, \bar{t}_j]$ (interval support), or $a(j) = \bar{t}_j$ under an end-of-window convention (timestamp support). Pointwise stages in $\mathcal{G}$ (e.g., scaling) do not affect T' and are applied on whichever grid is current.

Target alignment via raw-time mapping Akin to the feature-to-raw-time mapping defined above, target alignment is treated by associating each transformed feature index j with a raw-time support $a(j)$. The target at index j is then constructed from the raw target trajectory $\mathcal{Y}$ using this support, ensuring that each transformed feature is paired with a well-defined supervision signal. This design decouples feature extraction from supervision design, thereby supporting arbitrary feature transformations, including those that alter temporal resolution, while preserving a consistent definition of the target under different alignment conventions.

Because $\mathcal{G}$ may change the number and meaning of time steps, the target pipeline must produce one label per transformed index j . We therefore allow the target pipeline to depend on the raw target series and the associated support $a(j)$. This explicit alignment is particularly critical for prognostics tasks, where targets such as Remaining Useful Life depend on precise temporal positioning (e.g., assigning the RUL at the end of a window) and scaling, whereas in diagnostics tasks labels are often constant or slowly varying over intervals and are therefore less sensitive to the exact alignment choice:

$$z_y(j) = \tilde{\mathcal{H}}(\mathcal{Y}, a(j); \Phi), \quad j = 1, \dots, T'. \quad (8)$$

Here, $\tilde{\mathcal{H}}$ is a composition of Q stages (e.g., clipping, scaling, calibration) and may include the previously defined alignment/aggregation operator $\mathcal{A}$ at an arbitrary position in the composition. A common special case is

$$z_y(j) = \mathcal{A}(\mathcal{H}(\mathcal{Y}; \Phi), a(j)), \quad (9)$$

but other transformation sequence orderings (e.g., transforming $y(t)$ pointwise before aggregation over $a(j)$) are equally admissible and task-dependent. Concretely, $\mathcal{A}$ may implement pointwise sampling $\mathcal{A}(\mathcal{Y}; a(j)) = y(a(j))$ when $a(j)$ is a timestamp, or window aggregation (e.g., mean/last/max/majority) over $\{y(t)\}_{t \in a(j)}$ when $a(j)$ is an interval. In our experiments, unless otherwise stated, we use an end-of-window convention for window-derived features (i.e., $a(j) = \bar{t}_j$).

Target alignment yields the aligned target series $\mathcal{Y}' = \{z_y(j)\}_{j=1}^{T'}$ paired with $\mathcal{Z}$. After this stage, the target alignment remains fixed and is no longer modified.

Leakage Policy. All fitted parameters Ψ and Φ (and any statistics used by $\mathcal{A}$, e.g., thresholds or calibration) are estimated exclusively using the training partition. Once estimated, they are frozen and applied unchanged to validation and test partitions.

Example. The feature pipeline $\mathcal{G}$ is a two-stage composition. The first stage g_1 applies min-max normalization channel-wise using training-partition statistics and preserves the current temporal grid. The second stage g_2 computes K time-domain statistics—mean, maximum, RMS, standard deviation, and others—over a sliding window of width w raw samples, expanding the M raw channels into $F = MK$ summary features at each transformed index j . The support of $\mathbf{z}(j)$ is therefore the interval $a(j) = [\bar{t}_j - w + 1, \bar{t}_j]$. For target alignment, we adopt an end-of-window convention, so $a(j) = \bar{t}_j$ for supervision and $z_y(j) = \tilde{\mathcal{H}}(\mathcal{Y}, \bar{t}_j; \Phi)$. In this example, the target transformation inside $\tilde{\mathcal{H}}$ scales ah-RUL by dividing by the maximum value observed in the training partition, with this scaling constant included in Φ . Because ah-RUL has a natural lower bound of zero at end-of-life but no fixed upper bound, a cell near end-of-life yields $z_y(j) \approx 0$, while a fresh cell yields $z_y(j) > 0$ and may yield $z_y(j) > 1$ for test cells with longer lifetimes than any training cell.

3.1.3. Sequence Slicing (S)

Following the pre-processing stage, the goal of this stage is to construct supervised learning samples for predictive modeling by forming the final input–target pairs from the transformed feature sequence. Rather than predicting from a single time step, each input consists of a finite history window of length L_{seq} (i.e., how much past context is provided), which is paired with a corresponding target value derived from the aligned target sequence. The feature window is created by the slicing operator S and is applied to the transformed feature sequence $\mathcal{Z} = \{\mathbf{z}(j)\}_{j=1}^{T'}$. Note that this bound on the window length is imposed on the data-level by the input representation and may differ from the effective lookback utilized at the model level due to architectural or optimization constraints. To formalize the construction of S , we introduce a set of additional parameters besides L_{seq} that control how windows are extracted:

- $\Delta \in \mathbb{N}$: the stride between consecutive windows (controls overlap and dataset size),
- $\rho \in \mathbb{Z}_{\geq 0}$ (left-padding allowance, with $\rho = 0$ strict windowing): an optional warm-start depth that allows windows to start before the first valid index via padding,
- $\delta \in \mathbb{Z}_{\geq 0}$: a supervision offset that determines how far into the future (relative to the window end) the target is read,

- $L_{\text{pred}} \in \mathbb{N}$: the length of the supervision segment, where $L_{\text{pred}} = 1$ corresponds to a single window-level label.

In the PHM tasks considered here, we use window-level prediction with $L_{\text{pred}} = 1$, i.e., a single target value per window and typically $\delta = 0$, meaning the target corresponds to the end of the window. However, the more general formulation allows for multi-step prediction or auxiliary objectives without modifying the preprocessing pipeline.

Valid start indices and dataset size. To determine the final dataset length, we now characterize the set of valid start indices for window extraction on the transformed timeline, given $(L_{\text{seq}}, \Delta, \rho, \delta, L_{\text{pred}})$ and transformed length T' .

$$L_{\text{req}} \triangleq L_{\text{seq}} + \delta + L_{\text{pred}} - 1. \quad (10)$$

$$k_m \triangleq 1 - \rho + (m - 1)\Delta, \quad m = 1, 2, \dots \quad (11)$$

$$k_m + L_{\text{req}} - 1 \leq T'. \quad (12)$$

$$N_{\text{slices}} \triangleq \max \left\{ 0, \left\lfloor \frac{T' - L_{\text{req}} + \rho}{\Delta} \right\rfloor + 1 \right\}, \quad \mathcal{K} \triangleq \{k_m\}_{m=1}^{N_{\text{slices}}}. \quad (13)$$

Here, L_{req} is the required right-side coverage in transformed time under the chosen supervision configuration; k_m enumerates candidate window starts with stride Δ and warm-start depth ρ (so the earliest start is $1 - \rho$); the inequality enforces that each window has sufficient remaining trajectory length; N_{slices} is the resulting number of admissible windows; and $\mathcal{K}$ is the set of admissible start indices.

Left-padding for the feature history (abstract). One challenge when comparing models with different context window sizes is that, without additional measures, changing L_{seq} alters the (testing) dataset size, leading to unfair comparisons between models. To address this, we introduce left padding. If $\rho > 0$, some history indices may satisfy $k_m + i < 1$. We therefore define a left-padded feature extension via an abstract padding operator $\mathcal{P}$:

$$\tilde{\mathbf{z}}(j) = \begin{cases} \mathbf{z}(j), & j \geq 1, \\ \mathcal{P}(\mathbf{z}, j), & j \leq 0. \end{cases} \quad (14)$$

For each admissible start $k_m \in \mathcal{K}$, the feature window is

$$\mathbf{W}_m = [\tilde{\mathbf{z}}(k_m), \tilde{\mathbf{z}}(k_m + 1), \dots, \tilde{\mathbf{z}}(k_m + L_{\text{seq}} - 1)]^\top \in \mathbb{R}^{L_{\text{seq}} \times F}. \quad (15)$$

For window-level supervision, the supervision index and label are

$$j_{\text{sup}}(k_m) = k_m + L_{\text{seq}} - 1 + \delta, \quad y_m = z_y(j_{\text{sup}}(k_m)). \quad (16)$$

The label is guaranteed to come from a real (non-padded) target index for all m provided $\rho \leq L_{\text{seq}} - 1 + \delta$.

The resulting windowed dataset for a single unit is

$$\mathcal{D}_{\text{seq}} \triangleq \{(\mathbf{W}_m, y_m)\}_{m=1}^{N_{\text{slices}}}. \quad (17)$$

The output of this stage is the set of temporal windows $\mathcal{W} = \{\mathbf{W}_m\}_{m=1}^{N_{\text{slices}}}$ paired with labels $\{y_m\}_{m=1}^{N_{\text{slices}}}$.

Example. Each extracted window $\mathbf{W}_m = [\mathbf{z}(m), \dots, \mathbf{z}(m + L_{\text{seq}} - 1)]^\top \in \mathbb{R}^{L_{\text{seq}} \times F}$ stacks L_{seq} consecutive feature vectors and is paired with the ah-RUL label $y_m = z_y(m + L_{\text{seq}} - 1)$ at the window end. With stride $\Delta = 1$, consecutive windows shift by one step, so labels decrease slowly as the battery ages. A time-series Transformer (or LSTM) consumes $\mathbf{W}_m$ directly as its input sequence, treating each row as one time step.

3.1.4. Tabularization Schema ($\mathcal{T}$)

To support heterogeneous model classes, we define two representations of each sample. Sequence models (e.g., LSTMs and Transformers) operate directly on the temporal window $\mathbf{W}_m \in \mathbb{R}^{L_{\text{seq}} \times F}$. By contrast, tabular foundation models are pre-trained on row-wise tabular data, where each example is represented by a fixed-dimensional feature vector and inference is performed over sets of such rows. They therefore assume no explicit sequence axis in the input. For compatibility with this input schema, each window $\mathbf{W}_m$ is mapped to a vector representation that preserves all entries and encodes temporal order through a fixed column ordering.

We define the tabularization operator $\mathcal{T} : \mathbb{R}^{L_{\text{seq}} \times F} \rightarrow \mathbb{R}^{D_{\text{tab}}}$ to transform the sequence $\mathbf{W}_m$ into a flat tabular sample $\mathbf{X}_m$, where $D_{\text{tab}} = L_{\text{seq}} F$:

$$\mathbf{X}_m = \mathcal{T}(\mathbf{W}_m) = [z_1(k_m), \dots, z_F(k_m), \dots, z_1(k_m + L_{\text{seq}} - 1), \dots, z_F(k_m + L_{\text{seq}} - 1)]^\top, \quad (18)$$

where $z_f(j)$ denotes the f -th component of $\mathbf{z}(j)$ and we use a time-major ordering for reproducibility. The labels remain paired with the tabularized rows; only the input window is flattened by $\mathcal{T}$.

Example. The window $\mathbf{W}_m \in \mathbb{R}^{L_{\text{seq}} \times F}$ is flattened in time-major order to yield the tabular vector $\mathbf{X}_m = \mathcal{T}(\mathbf{W}_m) \in \mathbb{R}^{D_{\text{tab}}}$ with $D_{\text{tab}} = L_{\text{seq}} F$. The first F entries encode the windowed statistics at the earliest step in the window, and the last F entries encode those at the most recent step. The pair $(\mathbf{X}_m, y_m)$ is one element of $\mathcal{D}_{\text{tab}}$. At inference time, TabPFN or TabDPT conditions on an admissible training-only context set $\mathcal{C} \subset \mathcal{D}_{\text{tab}}^{\text{train}}$ to predict the ah-RUL of a query battery window.

3.1.5. Dual Dataset Representation

The application of the transformation pipeline yields two information-equivalent (up to reshaping) sample representations: the temporal window $\mathbf{W}_m$ and its tabularized counterpart $\mathbf{X}_m$. At the dataset level, these induce the sequence dataset $\mathcal{D}_{\text{seq}}$ and the tabular dataset $\mathcal{D}_{\text{tab}}$. The final dimension of the tabular vector is $D_{\text{tab}} = L_{\text{seq}} F$. One may then select the representation that matches the target model class:

- **Sequence Dataset:** $\mathcal{D}_{\text{seq}} = \{(\mathbf{W}_m, y_m)\}_{m=1}^{N_{\text{slices}}}$, which retains the explicit temporal matrix structure produced by applying $\mathcal{G}$ and slicing $\mathcal{S}$ to the raw data and using the aligned target series $\mathcal{Y}'$.
- **Tabular Dataset:** $\mathcal{D}_{\text{tab}} = \{(\mathbf{X}_m, y_m)\}_{m=1}^{N_{\text{slices}}}$, which extends the pipeline with the flattening operator $\mathcal{T}$, making the data structurally compatible with tabular learning algorithms.

These formats contain identical information content, allowing for flexible model selection without data loss. Both representations inherit the same admissible start set $\mathcal{K}$ and the same split membership for each sample. The total number of samples N_{slices} in both $\mathcal{D}_{\text{seq}}$ and $\mathcal{D}_{\text{tab}}$ is directly controlled by the slicing stride Δ . By adjusting Δ , we can effectively perform subsampling to manage computational constraints or reduce redundancy without altering the fundamental structure of the samples.

Context sets for tabular foundation models (leakage constraints). When using tabular foundation models with in-context learning, predictions may take the form $\hat{y}_m = f_\phi(\mathbf{X}_m; \mathcal{C})$, where $\mathcal{C} = \mathcal{D}_{\text{tab}}^{\text{train}}$ is the set of in-context samples. To prevent leakage and ensure comparability, $\mathcal{C}$ must be drawn exclusively from the training partition; no validation/test samples may appear in $\mathcal{C}$. Under intra-unit temporal splitting, context selection must additionally respect time, i.e., it must not use samples derived from future timestamps relative to the supervision index $j_{\text{sup}}(k_m)$ of the query sample. The context size and sampling/retrieval strategy are fixed in advance, pre-cached for increased performance and applied consistently across methods.

3.1.6. Generalization to Multiple Units and Partitions

For a dataset with U heterogeneous units, we apply the operators $\mathcal{G}$, $\mathcal{S}$, and $\mathcal{T}$ independently to each unit u . To learn across multiple units, we define the global dataset $\mathcal{D}_{\text{global}}$ as the union of these processed unit-specific subsets. Depending on the chosen representation (sequence or tabular), this is defined as:

$$\mathcal{D}_{\text{global}} = \bigcup_{u=1}^U \mathcal{D}_{\text{unit}}^{(u)} \quad (19)$$

where $D_{unit}^{(u)}$ is either $D_{seq}^{(u)}$ or $D_{tab}^{(u)}$.

We formally define the partitioning of the global dataset into disjoint training, validation, and testing subsets:

$$D_{global} = D_{global}^{train} \cup D_{global}^{val} \cup D_{global}^{test} \quad (20)$$

This partitioning can be performed using two distinct strategies depending on the application constraints:

- **Inter-Unit Splitting:** The set of units $\{1, \dots, U\}$ is disjointly divided. All samples derived from a specific unit belong exclusively to one partition. This tests generalization to unseen machines.
- **Intra-Unit Splitting:** The splitting occurs along the temporal dimension within each unit. This evaluates generalization across time on known machines under a causal protocol (no future information in training-time preprocessing).

Prevention of Data Leakage: Crucially, the transformation parameters $\Psi = \{\psi_1, \dots, \psi_P\}$ (introduced in Stage 1) are estimated *solely* from the training partition D_{global}^{train} . Likewise, any fitted target parameters Φ and any choices inside $\tilde{H}$ (including alignment/aggregation design and any statistics it requires) are determined using only the training partition. These fixed parameters are then applied to transform the validation and test partitions, ensuring no information from future or unseen data influences the feature extraction process. For intra-unit splitting, “training partition” refers to the training time range within each unit, so that no future timestamps contribute to fitted preprocessing statistics.

Due to the frameworks abstraction layers, models receive individual windows $\mathbf{W}_m$ identically regardless of which unit they originate from. However, the framework is unit-sensitive: unit identities are tracked throughout the pipeline to support per-unit evaluation (e.g., per-unit MAE on battery and bearing datasets; Appendix C.6.2). For clarity and without loss of generality, throughout the remainder of this work we refer to the datasets as D_{seq} or D_{tab} without explicit mention of the global split superscripts, unless the context requires distinguishing between partitions.

3.1.7. Task-Specific Target Definitions

The goal of the learning task is to map each input window $\mathbf{W}_m$ (or its tabular representation $\mathbf{X}_m$) to an associated target variable. The precise definition of this target depends on the specific PHM task and how supervision is aligned with the input window. In particular, targets may represent either future system evolution or the current system state, and can be defined at a specific time index or aggregated over the window.

- **Prognostics:** The target is a scalar $y_m \in \mathbb{R}_{\geq 0}$, representing a monotonically decreasing value such as the Remaining Useful Life (RUL) associated with the end of the window $\mathbf{W}_m$. Unless otherwise stated, supervision is read at the window-level supervision index $j_{sup}(k_m) = k_m + L_{seq} - 1 + \delta$, and the label is defined as $y_m = z_y(j_{sup}(k_m))$. The mapping from $j_{sup}(k_m)$ to raw time (and any windowing/aggregation) is governed by $a(j_{sup}(k_m))$ and the target pipeline $\tilde{H}$.
- **Diagnostics:** The target is a discrete class label $y_m \in \{0, 1, \dots, K - 1\}$, indicating the specific fault type present within the window $\mathbf{W}_m$. If the diagnostic label is defined over the window (rather than pointwise), this is implemented within $\tilde{H}$ by choosing $\mathcal{A}$ to aggregate labels over the supports corresponding to the input window (e.g., last-value, max-over-window, or majority vote), rather than implicitly redefining H .

4. Training and Implementation

4.1. Supervised Learning with Sequential Models

Conventional sequence models, such as 1D-CNNs, LSTMs, and Time-Series Transformers, are trained to minimize a task-specific loss over the entire sequence dataset D_{seq}^{train} . These models do not operate on the tabularized data $\mathbf{X}_k$; instead, they ingest the temporal sequence $\mathbf{W}_k$ directly.

Let f_θ denote a model parameterized by θ . The model produces a prediction $\hat{y}_k = f_\theta(\mathbf{W}_k)$. The optimal parameters θ^* are obtained by minimizing the empirical risk over the training sequence dataset:

$$\theta^* = \arg \min_{\theta} \frac{1}{|D_{seq}^{train}|} \sum_{(\mathbf{W}_k, y_k) \in D_{seq}^{train}} \ell(f_\theta(\mathbf{W}_k), y_k) \quad (21)$$

where $\ell(\cdot, \cdot)$ is a task-dependent loss function (e.g., Mean Squared Error for prognostics or Cross-Entropy for diagnostics). We typically employ stochastic gradient descent variants (e.g., AdamW) to solve this optimization problem.

4.2. Tabular Foundation Models: The In-Context Learning Paradigm

Tabular Foundation Models (e.g., TabPFN) fundamentally operate via In-Context Learning (ICL). In this paradigm, the model f_ϕ , parameterized by weights ϕ , makes predictions for a query instance $\mathbf{X}_q$ by conditioning on a context set C of labeled support examples. The prediction is given by:

$$\hat{y}_q = f_\phi(C, \mathbf{X}_q) \quad (22)$$

Crucially, to ensure a valid evaluation and prevent data leakage, the context set C must always be drawn strictly from the training partition of the tabular dataset:

$$C = \{(\mathbf{X}_j, y_j)\}_{j=1}^C \subset \mathcal{D}_{\text{tab}}^{\text{train}} \quad (23)$$

This mechanism allows the model to adapt its posterior distribution to the specific task at inference time. We utilize this general paradigm in two distinct modes:

4.2.1. Pre-trained In-Context Learning (Zero-Shot)

We use the foundation model with its original, pre-trained weights ϕ (frozen). The model adapts to the specific PHM task solely through the information provided in the context set C (drawn from $\mathcal{D}_{\text{tab}}^{\text{train}}$) at inference time.

$$\hat{y}_q = f_\phi(C, \mathbf{X}_q) \quad (24)$$

The context size is typically restricted (e.g., $|C| \leq 4 \times 10^4$). If the available training data in $\mathcal{D}_{\text{tab}}^{\text{train}}$ exceeds this limit, we construct C by subsampling the tabularized training windows.

For the data-efficiency experiments, we vary this subsampling explicitly through a subset ratio $r_{\text{sub}} \in (0, 1]$, applied after sequence construction and tabularization. With aggregate subsampling, $\max(1, \lfloor r_{\text{sub}} N \rfloor)$ windows are sampled uniformly without replacement from the N eligible training windows, and the selected indices are sorted before constructing the context or training set. With blockwise subsampling, the same total number of windows is distributed over B contiguous index blocks with randomly sampled block lengths and gaps. The selected windows therefore remain locally contiguous while covering only parts of the training trajectories. This operation reduces the number of rows supplied to the model, but it does not alter the feature dimension, sequence length, or contents of any selected window. Test windows are not subsampled in these experiments.

4.3. Evaluation Protocol

To ensure a fair and standardized comparison across disparate modeling architectures, we employ a unified evaluation protocol. Regardless of whether a model consumes sequential data ($\mathbf{W}_k \in \mathcal{D}_{\text{seq}}$) or tabular data ($\mathbf{X}_k \in \mathcal{D}_{\text{tab}}$), all metrics are calculated on the same set of test instances derived from the held-out partition $\mathcal{D}^{\text{test}}$.

We define a task-specific loss function $\ell(\hat{y}, y)$ (e.g., squared error for regression or 0-1 loss for classification). The aggregate test loss $\mathcal{L}_{\text{test}}$ is computed over the test set, ensuring a consistent evaluation logic for both model families:

$$\mathcal{L}_{\text{test}} = \frac{1}{|\mathcal{D}^{\text{test}}|} \sum_{k \in \mathcal{D}^{\text{test}}} \ell(\hat{y}_k, y_k), \quad \text{where } \hat{y}_k = \begin{cases} f_\theta(\mathbf{W}_k) & \text{if } f \text{ is Sequential} \\ f_\phi(C, \mathbf{X}_k) & \text{if } f \text{ is Tabular} \end{cases} \quad (25)$$

For Prognostics, we employ standard regression metrics such as Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE). For Diagnostics, we utilize classification metrics including Accuracy and Macro-F1 Score to account for class imbalance. For cross-dataset comparisons, we report the average rank obtained by ranking models within each dataset and averaging the resulting ranks across datasets, with rank 1 denoting the best model for a given dataset.

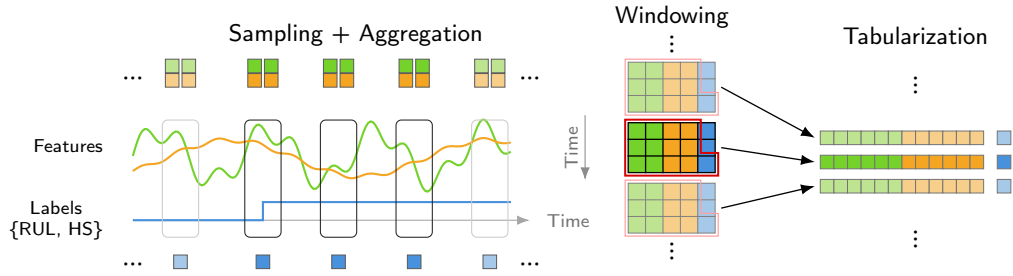


Figure 2: Our tabularization scheme. Time-series features are aggregated into their statistics on a per-window basis, along with the corresponding labels. These are then composed into a row in our table, providing richer context.

5. Practical Implementation Details

5.1. Unified Evaluation Pipeline Details

All baselines are trained from scratch and evaluated alongside all foundation models using the evaluation infrastructure of Telyatnikov et al. [52], under identical preprocessing, splitting, windowing, and metric computation (Section 4.3), ensuring that performance differences reflect modeling choices rather than data preparation. Model training is performed using PyTorch with its implementation of the AdamW optimizer. The framework uses an adapted version of the Trainer provided by Lightning¹, extended to accommodate models that use a Scikit-learn-like `fit()` and `predict()` API. Our training regimen for transformer models incorporates a custom learning rate scheduling that includes a warm-up phase and reduces the learning rate upon reaching a plateau, as this step has improved performance in the original forecasting tasks. We use min-max or z-score normalization, depending on the dataset. All preprocessing, feature extraction, and sequence slicing, represented by the operators $\mathcal{G}$ and $\mathcal{S}$, are shared among the models. The tabular foundation models and XGBoost share the same tabularization schema $\mathcal{T}$. Hyper-parameter optimization was performed using grid search, with initial parameter ranges selected from the original works. Hyper-parameter tuning, which includes the number of columns and rows for tabular in-context learning, was performed on the validation set. Dataset split and target-construction metadata are summarized in Section A (e.g., Tables 3, 4, 6, and 7). We report means and standard deviations computed over five random seeds.

5.2. Benchmark Models

This section introduces the model families considered in our study and formalizes the data-processing pipeline used to compare them fairly. Since our objective is to benchmark *tabular foundation models* against established *time-series models* on PHM tasks, the key challenge is to construct two representations of the same underlying problem: a sequence representation for temporal models and a tabular representation for foundation models. For completeness, we also include strong non-transformer baselines commonly used in PHM.

5.2.1. Tabular Foundation Models

Recent progress in Prior-Fitted Networks has led to a growing family of tabular foundation models (TFMs), including several alternatives to the original TabPFN framework (e.g., [45, 3]). In this work, we evaluate the following two representatives:

- **TabPFN:** Tabular Prior-Fitted Networks [26, 27], are designed to approximate Bayesian inference on tabular datasets. Trained on large-scale causally-generated synthetic data, TabPFN supports both classification and regression through in-context learning and often outperforms strong tree-based methods. More details are given in Appendix B.3.
- **TabDPT:** A transformer-based tabular foundation model trained on real-world datasets, that uses retrieval-based self-supervised pre-training and in-context learning to generalize to unseen tabular datasets, including both classification and regression, with no task-specific training or hyper-parameter tuning [38]. More details are given in Appendix B.3.

¹<https://lightning.ai/docs/pytorch/stable/>

5.2.2. Transformer models

For our experiments, we benchmark tabular foundation models against state-of-the-art (SOTA) models in long-range time-series forecasting. These models had to be adapted to perform regression and classification, as described in Section 3. Within the family of transformer-based models, these are:

- **PatchTST:** A channel-independent “patching” Transformer that tokenizes univariate time series into subseries-level patches, enabling longer receptive fields with lower attention cost [42].
- **Crossformer:** A Transformer for multivariate time series that tokenizes inputs with cross-dimensional embeddings and applies a two-stage attention layer to model both cross-time and cross-feature dependencies [64].
- **Spacetimeformer:** A long-range Transformer that jointly learns temporal and spatial interactions by treating spatiotemporal values as tokens, combining sequence and graph-like reasoning [23].

For completeness, we include non-transformer-based models in our comparisons. These encompass classical ML architectures such as CNNs and LSTMs [32], which have found wide adoption in the industry [34, 55], as well as statistical and tabular fit-predict baselines, and more recent models:

- **1D-CNN:** Convolution-based models that slide learnable kernels over temporal inputs to extract local trends, scale efficiently over long histories, and capture multi-resolution features through residual blocks.
- **LSTM:** Recurrent networks with gated memory cells that propagate and update a hidden state step-by-step, enabling nonlinear modeling of sequences with adaptive retention of long-term information. We implement the bi-direction variant, which integrates future covariates effectively [49]
- **Statistical regression baselines:** Linear, polynomial, and exponential regression are included as low-capacity reference models.
- **MLP:** A multilayer perceptron is included as a non-sequential neural baseline. It uses fully connected layers to model nonlinear interactions among the flattened temporal features.
- **XGBoost:** A classical tree-based baseline for tabular tasks. XGBoost uses gradient boosting with regularized decision trees and is evaluated on the same tabularized rows as the tabular foundation models [10].
- **TiDE,** a recently introduced dense residual model, built on MLP-based encoder-decoders and quasi-linear networks for long-term forecasting [11].

5.3. Adapting Baseline Models to Prognostics and Diagnostics tasks in PHM

The sequence models considered here—Bi-LSTM, 1D-CNN, TiDE, and all Transformer-based architectures—were originally designed for time-series forecasting and are applied directly to the sequential input $\mathbf{W}_k$. We adapt them to PHM regression and classification tasks by replacing their forecasting heads with task-specific regression or classification heads.

In addition, the original encoder decoder forecasting formulation does not directly apply to prognostics and diagnostics tasks in PHM. The past values of the target variable, namely the remaining useful life, are unavailable at inference time because the degradation and thus the time to failure are unknown prior to asset breakdown. Moreover, no future information is considered, as the objective is to regress onto the target at the same time index k . We therefore modify encoder-decoder transformer architectures by removing the encoder and directly feeding the sequential data $\mathbf{W}_k$ to the decoder. This step is not necessary for PatchTST, since it is a decoder-only transformer by design. However, its channel-independent processing cannot be applied to the regression target for the same reason. Therefore, we concatenate decoder latents and train a separate regression or classification head. TiDE introduces an attention-free multilayer perceptron (MLP)-based encoder-decoder architecture. Without lookback data, we disable TiDE’s encoder pathway as well and rely only on the model’s dynamic covariate pathway to process $\mathbf{W}_k$. Since all Transformer models produce point predictions for forecasting (auto-regressively), we use the same head for regression by simulating a prediction horizon of one. For diagnostics tasks, we replace the regression head with a classification head that outputs n classes.

Table 2

Main evaluation results. **Top block:** prognostics, measured by MAE in the normalized target space ($\times 100$) ($\downarrow$). **Bottom block:** diagnostics, measured by F1 score ($\times 100$) ($\uparrow$). Models are grouped by family: simple baselines, deep sequence models, transformers, tabular models, and tabular foundation models. **Bold/underline** denote the best/second-best result.

Model	NC-DS02 $\downarrow$	NC-P $\downarrow$	NB14 $\downarrow$	PHME20 $\downarrow$	Unibo $\downarrow$	XJTU-SY $\downarrow$	Avg rank
Linear	10.13 $\pm$ 0.14	16.11 $\pm$ 0.60	41.69 $\pm$ 12.02	12.19 $\pm$ 0.36	27.59 $\pm$ 14.36	76.80 $\pm$ 60.41	12.50
Exp	5.35 $\pm$ 0.06	10.96 $\pm$ 0.09	30.47 $\pm$ 47.76	8.82 $\pm$ 0.52	12.19 $\pm$ 0.31	27.22 $\pm$ 4.06	9.67
MLP	6.37 $\pm$ 0.23	13.17 $\pm$ 0.78	14.38 $\pm$ 9.77	4.62 $\pm$ 1.15	12.50 $\pm$ 0.76	30.64 $\pm$ 2.67	10.33
LSTM	4.93 $\pm$ 0.13	7.56 $\pm$ 0.31	3.80 $\pm$ 0.22	3.73 $\pm$ 0.98	6.50 $\pm$ 0.16	21.89 $\pm$ 0.40	3.67
CNN-1D	5.33 $\pm$ 0.37	7.53 $\pm$ 0.22	8.89 $\pm$ 1.70	5.35 $\pm$ 3.71	12.41 $\pm$ 1.15	31.02 $\pm$ 8.25	8.67
TiDE	5.29 $\pm$ 0.22	7.62 $\pm$ 0.20	3.44 $\pm$ 0.17	4.20 $\pm$ 0.66	6.46 $\pm$ 0.78	25.11 $\pm$ 2.38	5.17
TST	5.31 $\pm$ 0.13	7.02 $\pm$ 0.17	6.28 $\pm$ 0.25	4.11 $\pm$ 0.84	7.23 $\pm$ 0.39	33.30 $\pm$ 7.72	7.00
STF	4.89 $\pm$ 0.10	7.35 $\pm$ 1.16	10.67 $\pm$ 3.16	3.91 $\pm$ 1.00	8.89 $\pm$ 0.81	28.49 $\pm$ 4.01	6.17
CF	5.76 $\pm$ 0.51	9.98 $\pm$ 0.57	3.57 $\pm$ 0.07	3.87 $\pm$ 0.85	5.58 $\pm$ 1.08	22.09 $\pm$ 1.06	5.00
PTST	16.62 $\pm$ 0.04	21.55 $\pm$ 0.03	5.22 $\pm$ 0.10	15.09 $\pm$ 1.13	11.18 $\pm$ 1.11	25.42 $\pm$ 1.48	10.33
XGBoost	8.52 $\pm$ 0.00	15.24 $\pm$ 0.00	4.48 $\pm$ 0.00	2.68 $\pm$ 0.00	4.06 $\pm$ 0.00	24.59 $\pm$ 0.00	6.50
TabPFN	4.96 $\pm$ 0.04	7.79 $\pm$ 0.04	3.91 $\pm$ 0.03	1.95 $\pm$ 0.03	3.72 $\pm$ 0.06	22.27 $\pm$ 0.35	3.33
TabDPT	5.07 $\pm$ 0.06	6.85 $\pm$ 0.02	3.63 $\pm$ 0.04	2.19 $\pm$ 0.01	3.94 $\pm$ 0.05	23.24 $\pm$ 0.45	2.67

Model	NC-D $\uparrow$	HSF15-A $\uparrow$	HSF15-C $\uparrow$	HSF15-P $\uparrow$	HSF15-V $\uparrow$	MZVAV $\uparrow$	Avg rank
Linear	72.34 $\pm$ 3.04	58.75 $\pm$ 1.81	98.35 $\pm$ 0.82	54.40 $\pm$ 11.97	32.55 $\pm$ 2.36	39.89 $\pm$ 8.77	7.33
MLP	79.49 $\pm$ 1.80	91.02 $\pm$ 2.25	99.91 $\pm$ 0.14	97.32 $\pm$ 0.60	80.99 $\pm$ 29.38	60.10 $\pm$ 6.39	5.00
LSTM	88.84 $\pm$ 0.73	94.59 $\pm$ 0.97	100.00 $\pm$ 0.00	95.94 $\pm$ 2.83	97.35 $\pm$ 3.32	51.31 $\pm$ 6.01	3.83
CNN-1D	87.53 $\pm$ 2.83	94.03 $\pm$ 1.98	100.00 $\pm$ 0.00	98.73 $\pm$ 0.52	97.92 $\pm$ 0.83	66.11 $\pm$ 5.74	3.00
TiDE	32.57 $\pm$ 4.65	42.90 $\pm$ 5.07	61.57 $\pm$ 10.41	59.37 $\pm$ 7.68	42.11 $\pm$ 14.36	25.19 $\pm$ 5.29	8.17
TST	26.34 $\pm$ 3.96	37.37 $\pm$ 4.38	59.18 $\pm$ 10.29	46.07 $\pm$ 4.13	35.40 $\pm$ 5.77	24.93 $\pm$ 4.38	10.00
STF	24.55 $\pm$ 3.60	40.01 $\pm$ 5.75	65.94 $\pm$ 20.61	50.39 $\pm$ 11.54	37.34 $\pm$ 5.68	38.01 $\pm$ 5.56	8.67
CF	23.74 $\pm$ 0.93	25.04 $\pm$ 5.28	59.19 $\pm$ 10.05	29.46 $\pm$ 2.61	23.98 $\pm$ 2.95	17.34 $\pm$ 4.24	11.50
PTST	19.57 $\pm$ 0.26	31.56 $\pm$ 4.04	41.57 $\pm$ 5.18	41.22 $\pm$ 6.63	26.20 $\pm$ 2.44	25.81 $\pm$ 4.15	11.00
XGBoost	48.13 $\pm$ 0.00	98.07 $\pm$ 0.00	100.00 $\pm$ 0.00	99.66 $\pm$ 0.00	99.65 $\pm$ 0.00	57.08 $\pm$ 0.00	3.17
TabPFN	67.15 $\pm$ 1.46	99.47 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	58.32 $\pm$ 2.44	2.33
TabDPT	85.21 $\pm$ 0.16	96.66 $\pm$ 1.03	100.00 $\pm$ 0.00	99.06 $\pm$ 0.25	98.92 $\pm$ 0.35	71.29 $\pm$ 0.48	2.33

6. Results and Analysis

This section evaluates whether the proposed tabularization enables tabular foundation models to serve as a unified interface for PHM tasks. We compare tabular foundation models against multi-task baselines, including sequence models, transformer models, and a conventional gradient boosting framework, on prognostics and diagnostics benchmarks. We then analyze whether tabularizing PHM tasks is effective in practical scenarios by examining its behavior under limited training data, the role of temporal context retained in the tabular representation, and the structure of the predictive uncertainty produced by TabPFN. Full result tables, transformation schemas, and hyperparameter details are provided in Appendix C.

6.0.1. Overall Benchmark Performance

We first compare tabular foundation models across twelve diagnostics and prognostics scenarios. Table 2 reports performance for all 13 models across 12 datasets. Prognostics results are evaluated using normalized MAE ($\times 100$; top floor, $\downarrow$), while diagnostics results are evaluated using F1 score ($\times 100$; bottom floor, $\uparrow$), defined according to Section 4.3.

The prognostics results show a heterogeneous performance landscape: no single architecture dominates all datasets, and strong dataset-specific baselines remain competitive. Nevertheless, the tabular foundation models obtain the best average ranks, with TabDPT ranking first overall (average rank 2.67) and TabPFN second (average rank 3.33). A plausible explanation for this aggregate difference is that TabPFN’s synthetic pre-training priors are not designed specifically for PHM data. In contrast, TabDPT is pre-trained on real tabular datasets and constructs row-level contexts. This difference in pre-training and context construction may better match the heterogeneous row distributions induced by PHM tabularization. In addition to the best overall ranking, the tabular foundation models also perform best on certain datasets. TabPFN gives the lowest normalized MAE on PHME20 (1.95 ± 0.03) and Unibo (3.72 ± 0.06), while TabDPT gives the lowest normalized MAE on N-CMAPSS Prognostics (NC-P) (6.85 ± 0.02). Conventional sequence and transformer models remain strongest on selected tasks, with STF leading NC-DS02 (4.89 ± 0.10), TiDE leading NB14 (3.44 ± 0.17), and LSTM leading XJTU-SY (21.89 ± 0.40).

The diagnostic results show a similar pattern, with tabular foundation models achieving the best aggregate ranks. TabPFN and TabDPT obtain the best average rank across the six diagnostic tasks, with an average rank of 2.33 for

both. TabPFN achieves near-perfect Macro-F1 on the HSF15 component-level tasks, including HSF15-A, HSF15-C, HSF15-P, and HSF15-V where, interestingly, the performance of other transformer model is significantly lacking. TabDPT achieves the highest Macro-F1 on MZVAV. Deep sequence models remain competitive on NC-D, where LSTM and CNN-1D outperform both tabular foundation models. XGBoost remains a strong conventional fit-predict baseline on several HSF15 tasks, but is generally outperformed by the tabular foundation models. Overall, Table 2 supports the conclusion that tabular foundation models provide the most consistent cross-task performance, expressed through aggregate robustness rather than uniform dominance on every dataset.

6.0.2. Data Efficiency

Figure 5 evaluates prognostic performance on PHME20 and Unibo as the fraction of available training windows is varied under the protocol described in Section 4.2.1, using both aggregate random subsampling and blockwise subsampling. The PFN-based models are already competitive at very small data fractions, and TabPFN is particularly strong in the low-data regime. For PHME20, TabPFN and TabDPT achieve competitive performance using only 1% of the training data, with performance saturating once 10% of the data is available. On the Unibo data set, TabDPT requires 10% of the data to reach competitive performance, whereas TabPFN already performs competitively with 1%. Both PFN-based models again show a performance plateau at approximately 10% of the training data. Under random subsampling, the MAE decreases smoothly as additional windows are included, before reaching a saturation regime. This suggests that a relatively small but representative support set is sufficient for strong performance. In contrast, this behavior is not observed under blockwise subsampling, where the sampled data may fail to cover the test distribution. These results indicate that tabular foundation models are highly sensitive to the in-context data distribution. This context-sensitivity is further reflected in the main benchmark results: TabPFN’s performance on NC-DS02 is notably weaker than on NC-P, where the broader multi-source training distribution provides a richer and more representative context set. The same subset-ratio protocol is evaluated on MZVAV using Macro-F1, as shown in the bottom row of Fig. 5. Macro-F1 generally increases with larger data fractions, but the blockwise setting exhibits larger variance compared to the random subsampling for the PFN models. This indicates that blockwise subsampling changes the class coverage of the context more strongly than aggregate subsampling, which is consistent with the class-balance analysis in Fig. 6.

6.0.3. The Effect of Tabularizing Time Dependencies

Figure 7 shows that increasing the sequence length improves performance on PHME20, Unibo, and XJTU-SY, confirming that the tabularized representation can preserve task-relevant temporal information. On PHME20, TabPFN improves rapidly as the sequence length increases and then saturates, whereas TabDPT deteriorates after short histories. On Unibo, TabPFN remains consistently below TabDPT across the evaluated sequence lengths, with only limited additional gains after the shortest nontrivial histories. On XJTU-SY, TabPFN benefits substantially from longer histories, while TabDPT is less monotonic but outperforms TabPFN in absolute MAE. These patterns indicate that TabPFN is better able to exploit temporal context when history carries degradation information, which is consistent with the model’s cell-wise embedding of the flattened sensor-time grid, allowing TabPFN to extract meaningful causal temporal relations. TabDPT embeds complete rows and is therefore less directly aligned with temporal locality after flattening. For N-CMAPSS DS02, neither model benefits from longer histories; TabPFN becomes worse as sequence length increases, and TabDPT shows only small non-monotonic variation.

This behavior is consistent with the structure of N-CMAPSS. Degradation is modeled primarily at the flight cycle scale, while high frequency variation within a flight is dominated by changing operating conditions, e.g. the altitude. Consequently, extending the history window over consecutive samples from the same flight may add limited degradation information. Capturing meaningful temporal degradation trends in N-CMAPSS requires context over many flight cycles rather than only adjacent samples, which is beyond the sequence lengths evaluated here.

6.0.4. Data Imputation

We evaluate two strategies for handling missing values. The first applies last-observation-carried-forward (LOCF) imputation to all models uniformly: missing feature entries are filled with the most recent observed value before any model receives the input. The second passes the raw NaN-valued features directly to TabPFN v2, which handles missing values internally: NaN entries are replaced by the training-partition feature mean and a per-feature binary missingness indicator is appended as extra input features. The pre-trained transformer has seen this missingness representation during training and can therefore condition its predictions on the missingness pattern. All other models receive the

LOCF-imputed inputs, as they do not support NaN-valued inputs. We refer to the two TabPFN variants as *TabPFN* (LOCF-imputed) and *TabPFN+NaN* (TabPFN-internal handling) respectively.

Figure 3 reports the effect of missing data on normalized MAE for PHME20. TabPFN remains the strongest model under this missing-data setting, followed by TabDPT, indicating that the tabular foundation models retain their advantage when incomplete inputs are present. Unexpectedly, however, the explicit TabPFN NaN-token variant underperforms the imputed TabPFN configuration, indicating that TabPFN’s proposed missing-value handling strategy is not always effective. For this dataset and preprocessing pipeline, conventional imputation appears to provide a more informative representation than passing missingness directly to the model.

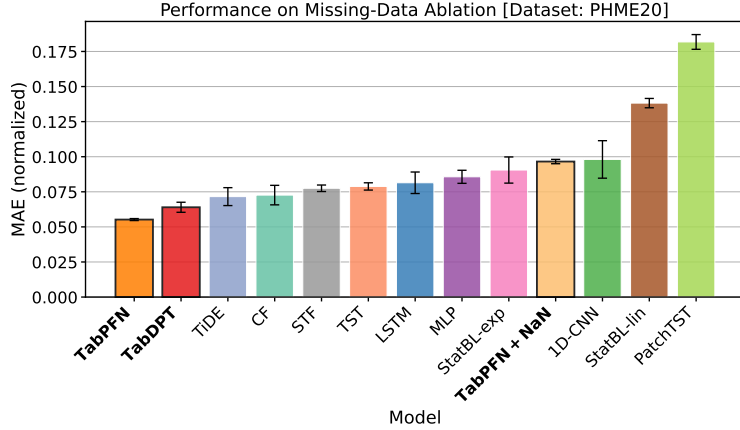


Figure 3: Effect of imputation on normalized MAE for PHME20. TabPFN+NaN indicates that we use the Nan token of TabPFN to represent missing values, while TabPFN indicates that we impute missing values.

6.0.5. Probabilistic Interpretation of the TabPFN Output

TabPFN represents continuous regression targets through a discretized predictive distribution rather than a single scalar; quantiles and point estimates are derived from this distribution. In Fig. 4 we interpret this predictive distribution qualitatively, examining how probability mass concentrates across RUL values as a function of time. In both datasets, the probability mass concentrates along repeated descending trajectories, indicating that the model recovers the monotonic run-to-failure structure of individual test units. For PHME20, the 20-step lookback produces more localized probability bands, whereas the one-step setting is more diffuse with broader vertical spread, particularly in early- and mid-life regions. For Unibo, both settings show descending trajectories and pronounced probability mass near the low-RUL endpoint. Similar to PHME20, the longer lookback localizes the bands more tightly along the descending trajectory. These qualitative observations indicate that temporal context sharpens the predictive distribution, although they do not constitute a formal calibration analysis.

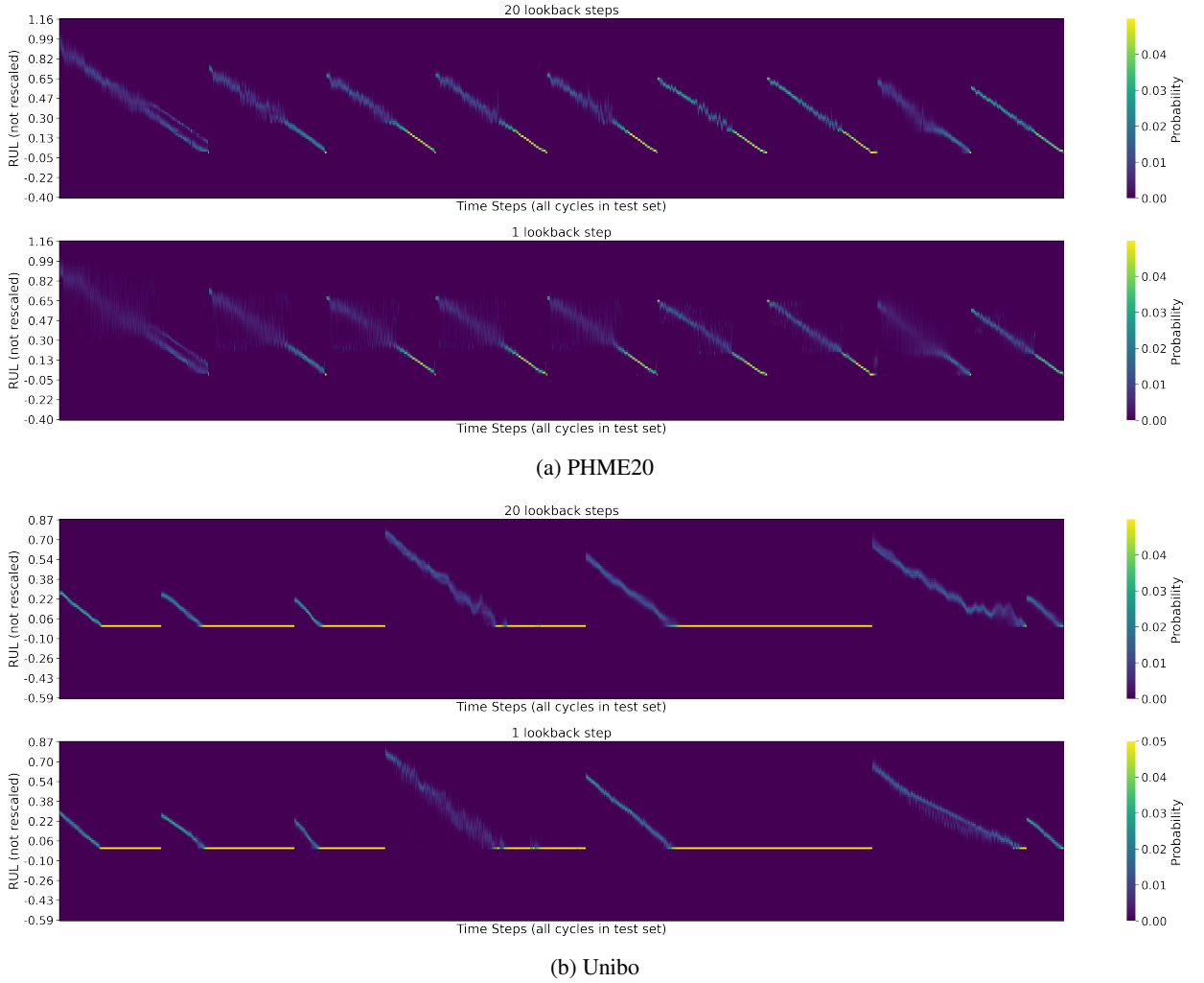
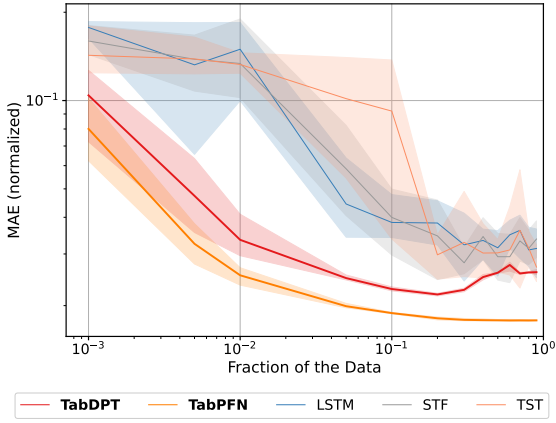
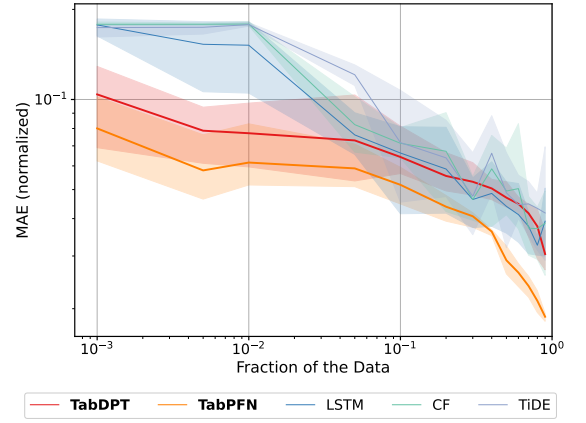


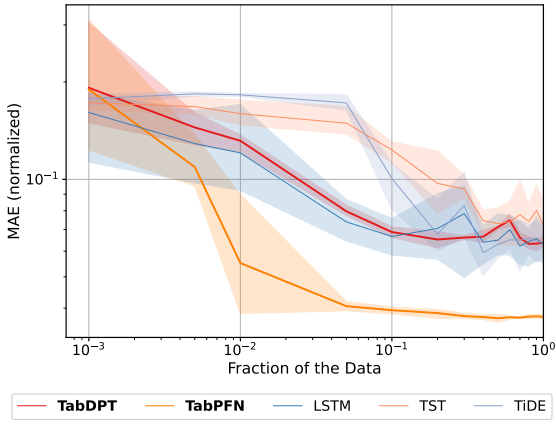
Figure 4: Predictive quantile distribution heatmaps for PHME20 and Unibo. RUL distribution with and without tabularization of time.



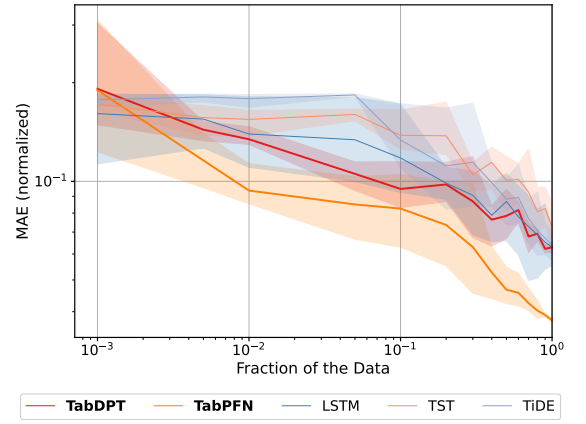
(a) PHME20



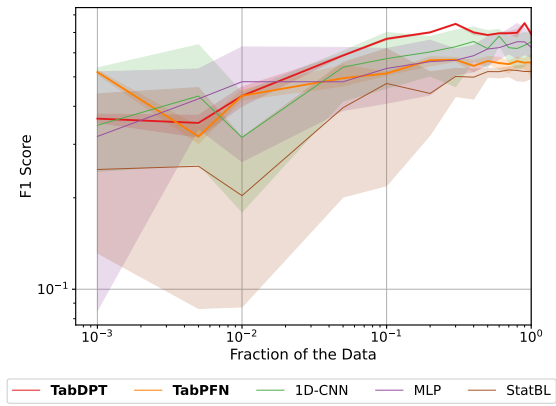
(b) PHME20, blockwise



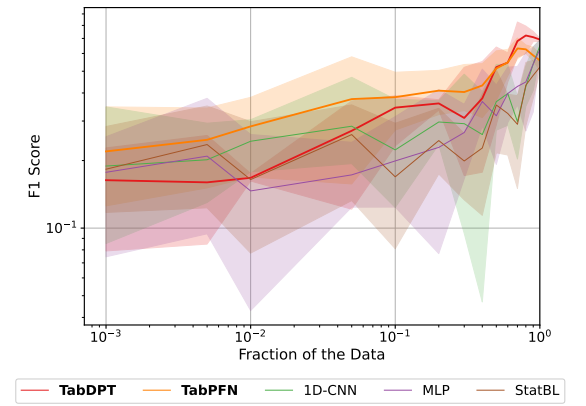
(c) Unibo



(d) Unibo, blockwise

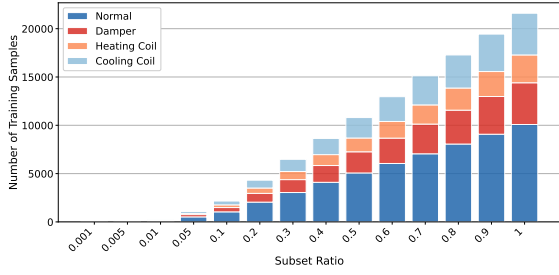


(e) MZVAV

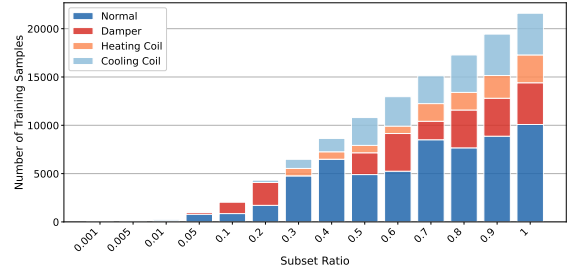


(f) MZVAV, blockwise

Figure 5: Scaling behavior for PHME20, Unibo, and MZVAV. The left column shows uniformly subsampled scaling laws; the right column shows blockwise subsampled scaling laws. We show the top 5 models for each dataset, which include TabPFN and TabDPT in all cases. A complete version with all models is included in the Appendix as Fig. 8.

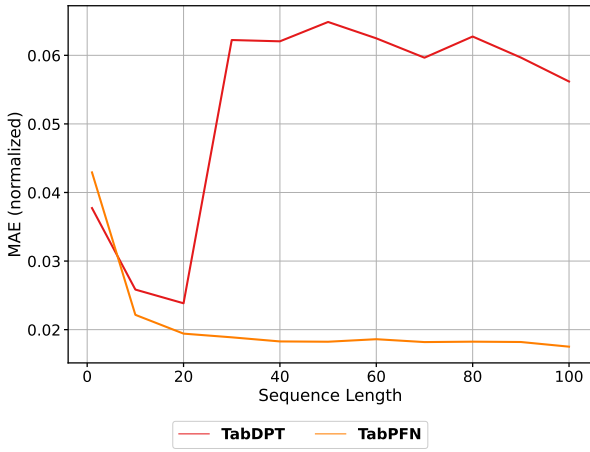


(a) Aggregate

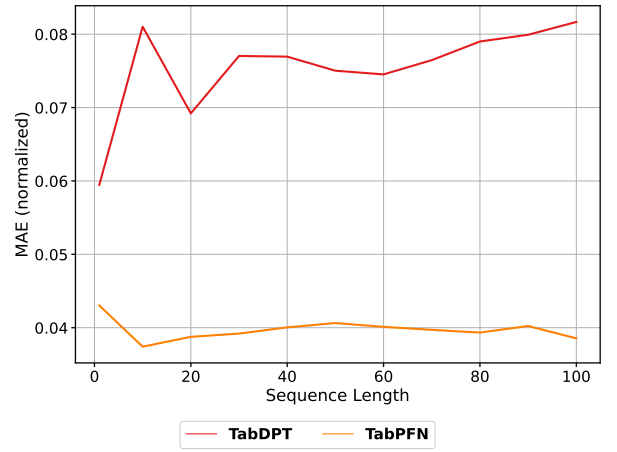


(b) Blockwise

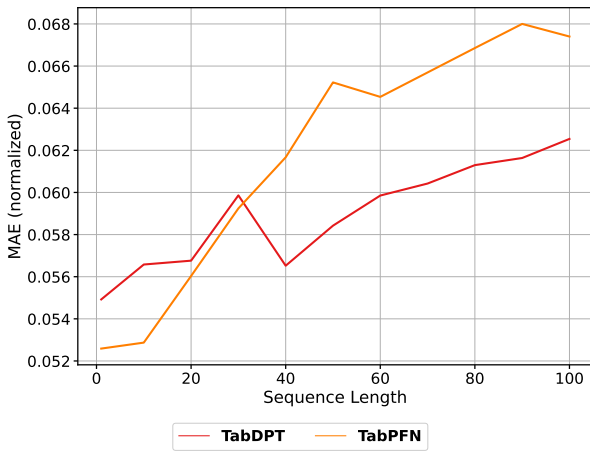
Figure 6: Class balance in the MZVAV subset-ratio experiment, comparing uniformly subsampled and blockwise subsampling (Seed: 72).



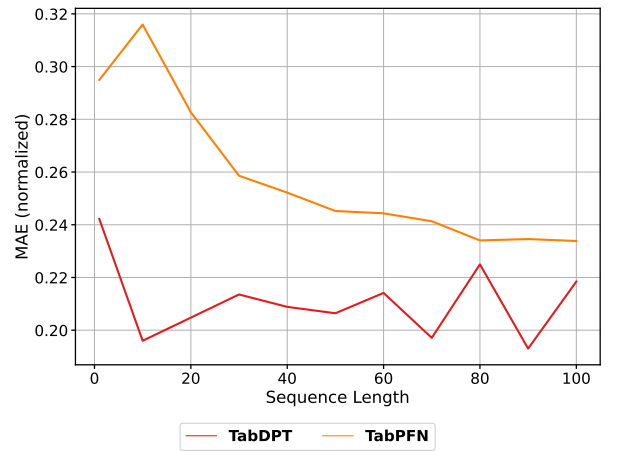
(a) PHME20



(b) Unibo



(c) N-CMAPSS DS02



(d) XJTU-SY

Figure 7: Effect of sequence length on normalized MAE for selected prognostics datasets.

7. Conclusion & Discussion

Our experiments demonstrate that tabularization acts as a flexible and effective framework for PHM tasks, allowing prognostics and diagnostics to be performed under a single representation. Additionally, our experiments indicate that our tabularization scheme enables Tabular Foundation Models (TFMs) to achieve strong performance with limited hyperparameter tuning concentrated mainly on the tabular shape. The results further show that TFMs exhibit the generalization capacity and data efficiency expected of foundation models, achieving the best average ranks without task-specific training and transferring effectively to new tasks through in-context learning.

Across both prognostics and diagnostics benchmarks, TFM remain highly competitive even under severe data scarcity and in the presence of missing values — with or without explicit imputation. This combination of sample efficiency, cross-task robustness, and few-shot generalization makes them attractive for industrial settings where labeled data is scarce and failure modes are heterogeneous.

This performance, however, depends critically on the distributional richness of the in-context table. Because these models learn from the provided context at inference time, the quality and coverage of that context directly determines prediction quality. Therefore, a context that does not cover the operating regimes, degradation stages, or fault modes present in the test data will result in degraded predictions. The N-CMAPSS families provide direct evidence of this: NC-DS02 is restricted to a single N-CMAPSS dataset file, with a limited set of units and failure-mode configurations, whereas NC-P pools multiple N-CMAPSS sources and fault configurations, yielding a richer and more varied training distribution. Notably, TabDPT achieves the best normalized MAE on NC-P among all evaluated models, underlining that tabular foundation models can excel when context diversity is sufficient. This distributional perspective is also consistent with the sequence-length analysis on NC-DS02, where additional temporal context provides only minimal benefit. Once the inference-time distribution is sufficiently covered, adding more samples or variables appears to provide little additional information for the model’s posterior. Taken together with the data-efficiency results, this finding is practically encouraging: because tabular foundation models can exploit small but well-chosen context sets, providing even a modest number of targeted samples that cover rare operating regimes or fault modes may be sufficient to unlock strong generalization in those settings.

More generally, PHM performance is strongly influenced by the preprocessing pipeline, making the choice of input transformations a central component of the evaluation protocol. The transformations selected in our experiments span both time and frequency domains to provide a comprehensive representation of the input signal. Since the primary objective of this study is cross-task robustness and sample efficiency rather than exhaustive per-dataset optimization, the shared transform set is intentionally conservative. This ensures fair comparison across model families rather than maximizing performance on any individual benchmark. Identifying more general transformation sets that transfer robustly across PHM datasets, architectures, and operating regimes remains an important direction for future work.

TFM’s computational efficiency is governed by the number of in-context samples and by the dimensionality of each tabularized row. In the present experiments, subsampling is applied only at the window level, and selected context rows retain the full flattened sensor-time representation. The finite inference-time budget therefore constrains the number of context samples that can be used, which can limit attainable accuracy. The current tabularization remains restrictive, as it does not allow individual sensors or time steps to be subsampled within a window. Extending the approach with per-variate or per-timestep subsampling could reduce D_{tab} , alleviate the computational bottleneck, and preserve more context samples by retaining only the most informative features. Future work may also develop more advanced context-selection strategies that preserve operating-regime, degradation-stage, and class coverage while remaining within the computational limits of TFM inference.

A significant advantage of PFN-based models is the dual utility of the validation set. In traditional deep learning, the validation set is strictly partitioned for hyperparameter tuning and early stopping to mitigate overfitting, effectively isolating it from the final inference phase. In contrast, PFNs allow the validation data to be repurposed. Once the optimal configuration (e.g., tabularization parameters) is identified, the validation set can be integrated into the model’s context. This enables the model to leverage a richer set of labeled points during inference on the test set, maximizing the informative value of all available data. A systematic study of how validation-set reuse interacts with context size, distributional shift, and tabularization choices remains an open and practically relevant research direction.

Finally, scalable PHM modeling may not be limited to replacing conventional methods with foundation models. Recent agentic PHM systems and benchmarks provide a complementary direction by automating parts of model configuration, tool orchestration, method reproduction, and evaluation under executable interfaces [9, 18, 53]. Such systems may make conventional methods more viable at scale by reducing the manual effort required to translate

paper-specific assumptions or user-defined PHM tasks into standardized preprocessing, model, target, and evaluation configurations.

Appendix

Appendix Contents

A. Dataset Protocols	24
Battery datasets	24
Bearing datasets	26
N-CMAPSS DS02 and multi-source families	27
Hydraulic diagnostics (HSF15)	27
PHME20	27
MZVAV	28
B. Implementation and Model Details	28
Sequential models and in-context tabular models	28
Evaluation protocol and baseline adaptation	29
TabPFN and TabDPT details	29
C. Additional Experimental Results	31
Experimental setup	31
Transformation schemas	31
Hyperparameter search	32
Reading the result tables	33
Diagnostics results	33
Prognostics results	34
Data-efficiency scaling	36
D. Reproducibility	40
Framework-based execution	40
Configuration-fixed experiments	40
Shared preprocessing and evaluation boundaries	40
E. Data and Code Access	40
Code and protocol availability	40
Third-party datasets	40

A. Dataset Protocols

This section records the dataset contract behind the main benchmark table. For each evaluated family, it states the source data, split logic, target definition, normalization, and metrics needed to interpret the reported prognostics or diagnostics results.

A.1. Battery Datasets

The battery prognostics tasks use NB14 and UNIBO21. Both datasets are treated as run-to-failure battery degradation problems with an operation-dependent RUL target based on remaining cumulative discharge.

NB14. Description. The NASA Randomized Battery Usage dataset [5] consists of 28 lithium cobalt oxide 18,650 cells organized into 7 operational groups. Battery aging is induced through repeated charge and randomized discharge cycles with loading periods of 5 minute. After 1500 loading periods, reference characterization cycles are used to assess battery health in terms of capacity. The signals are recorded at 0.1 Hz and comprise voltage, current, and temperature recorded during the aging tests.

Split. Following Bosello et al. [6], Group 3 is excluded, and cells RW3 and RW20 are removed because of corrupted or unusable measurements. The remaining 22 cells are split into training, validation, and test units; the exact split is shown in Table 3.

- **RW3** is excluded because of corrupted temperature measurements.
- **RW20** is excluded because its sensor data report zero for nearly its entire life.

Target, normalization, and metrics. NB14 uses the shared battery ah-RUL target defined below. Input features are min-max normalized, and performance is reported with MAE, MSE, and RMSE on Q_{RUL} .

UNIBO21. Description. The UNIBO Powertools dataset [57] contains laboratory data from 30 lithium-ion batteries used to study batteries in cleaning-equipment conditions. The cells differ by manufacturer, type, capacity, and test regime and can be categorized in 7 groups. The experiment protocol comprises a charging with 1.8 A current to a cutoff-voltage of 4.2 V, and discharging with a 5 A current until a defined end-of-discharge voltage. The health indicators

Table 3

Train/Validation/Test Split of the NASA Randomized Battery Dataset.

Group	Original Cells	Training Set	Validation Set	Test Set
Group 1	RW1, RW2, RW7, RW8	RW1, RW2	RW7	RW8
Group 2	RW3, RW4, RW5, RW6	RW4 (<i>RW3 excl.</i>)	RW5	RW6
Group 3	RW9, RW10, RW11, RW12	<i>Excluded (unrealistic profile)</i>		
Group 4	RW13, RW14, RW15, RW16	RW13, RW14	RW15	RW16
Group 5	RW17, RW18, RW19, RW20	RW17 (<i>RW20 excl.</i>)	RW18	RW19
Group 6	RW21, RW22, RW23, RW24	RW21, RW22	RW23	RW24
Group 7	RW25, RW26, RW27, RW28	RW25, RW26	RW27	RW28
Total	28 Batteries	10 Batteries	6 Batteries	6 Batteries

Table 4

Train/Validation/Test Split of the UNIBO21 Dataset.

Group	Original Cells	Training Set	Validation Set	Test Set
DM-3.0-S	000, 001, 002, 003	000, 001	002	003
DM-3.0-H	009, 010, 011	009	010	011
DM-3.0-P	013-017, (047, 049)	014, 015, 017	016	013
EE-2.85-S	006, 007, 008, 042	007, 008	042	006
EE-2.85-H	043, 044	043	—	044
DP-2.00-S	018, 036-039, 050, 051, (019)	018, 036, 037, 038, 050	051	039
DM-4.00-S	040, 041	040	—	041
Total	27 (+3 excl.)	15 Batteries	5 Batteries	7 Batteries

capacity and resistance were measured by performing 100 reference cycles. The collected signals are sampled with 0.1 Hz and include temperature, voltage, current, and energy measured during charging as well as discharging.

Split. Following Bosello et al. [6], cells 019, 047, and 049 are excluded because of data corruption or incomplete end-of-life cycling. The remaining 27 cells are split into training, validation, and test units, with the exact split shown in Table 4.

1. **019:** corrupted data.
2. **047, 049:** not cycled to end-of-life at the time of dataset construction.

Target, normalization, and metrics. UNIBO21 uses the same ah-RUL formulation as NB14, with dataset-specific implementation details for end-of-life and current integration. Input features are min-max normalized, and performance is reported with MAE, MSE, and RMSE on Q_{RUL} .

Target creation and metrics. Following Bosello et al. [6], both battery datasets use normalized remaining cumulative discharge, or ah-RUL, instead of a purely time- or cycle-based target. For any cycle n , ah-RUL is computed as:

$$Q_{RUL}(n) = Q_{acc}(n_{EoL}) - Q_{acc}(n) \quad (26)$$

where $Q_{RUL}(n) = 0$ for cycles $n \geq n_{EoL}$. The cumulative discharge throughput is computed by integrating discharge current and normalizing by nominal capacity:

$$Q_{acc}(n) = \frac{1}{Q_{nom}} \sum_{i=1}^n \left(\int_{t_i}^{t_{i+1}} I_d(t) dt \right) \quad (27)$$

The NB14 and UNIBO21 implementations differ in the definition of n_{EoL} and in how discharge current is selected, so the dataset-specific target rules are stated together with the split definitions.

Table 5

XJTU-SY Operating Conditions.

Condition ID	Engine Speed (rpm)	Radial Load (N)
Condition 1	2100	12,000
Condition 2	2250	11,000
Condition 3	2400	10,000

Table 6

XJTU-SY bearing lifetimes (min), used for regression targets and HI normalization.

Condition	Bearing Name	Bearing Lifetime (min)
Condition 1	Bearing 1_1	123
Condition 1	Bearing 1_2	161
Condition 1	Bearing 1_3	158
Condition 1	Bearing 1_4	122
Condition 1	Bearing 1_5	52
Condition 2	Bearing 2_1	491
Condition 2	Bearing 2_2	161
Condition 2	Bearing 2_3	533
Condition 2	Bearing 2_4	42
Condition 2	Bearing 2_5	339
Condition 3	Bearing 3_1	2538
Condition 3	Bearing 3_2	2496
Condition 3	Bearing 3_3	371
Condition 3	Bearing 3_4	1515
Condition 3	Bearing 3_5	114

A.2. Bearing Datasets

The evaluated bearing prognostics task is XJTU-SY. It provides high-frequency vibration measurements from accelerated run-to-failure bearing experiments and is evaluated as a health-indicator regression task.

XJTU-SY. Description. XJTU-SY, introduced by Yaguo et al. [61], provides 15 run-to-failure bearings under three operating conditions. Each bearing is recorded through horizontal and vertical high-frequency vibration channels until failure with a sampling rate of 25.6 kHz. In this benchmark, each acquisition is represented as a two-channel vibration segment of shape (32768, 2), and the operating conditions and bearing lifetimes used for target construction are summarized below.

Preprocessing. The preprocessing pipeline standard-scales the raw vibration channels, computes a cumulative-sum feature, extracts time-domain and spectral descriptors, and min-max rescales the concatenated descriptor block. Fit-predict models reuse the same processed representation through a tabularized variant.

Split. The reported XJTU-SY experiments use the configured PHMD split. The training bearings are 1_3, 1_4, 2_1, 2_4, 2_5, 3_1, 3_2, and 3_3; the validation bearings are 1_1, 1_2, and 3_5; and the test bearings are 1_5, 2_2, 2_3, and 3_4.

Target, normalization, and metrics. Runtime is converted into the HI target below using the bearing lifetime table. XJTU-SY is evaluated with standard regression metrics such as MAE and RMSE between the true HI and the predicted HI for each sample.

Target creation and metrics. For XJTU-SY, the target variable is formulated as a normalized health indicator (HI), linearly decreasing from 1 for a healthy bearing to 0 at failure:

$$HI = 1 - \frac{\text{Runtime}}{\text{Total Lifetime}} \quad (28)$$

For XJTU-SY, Total Lifetime is the Bearing Lifetime (min) in Table 6.

Table 7

N-CMAPSS-02 data splits, flight classes, and fault types (notation aligned with the main text).

Split	Unit number	Flight class
Train	2	2
Train	5	2
Train	10	2
Train	16	2
Train	20	2
Validation	18	2
Test	11	2
Test	14	2
Test	15	2

A.3. N-CMAPSS DS02 and Multi-Source Families

N-CMAPSS. Description. N-CMAPSS [4, 20] is a NASA benchmark family derived from C-MAPSS turbofan engine simulations. It provides multivariate time series with run-to-failure trajectories, operating conditions, fault modes, and ground-truth RUL. This work uses this source family in two scopes: NC-DS02 denotes the DS02 prognostics protocol, while NC-P and NC-D denote the broader N-CMAPSS prognostics and diagnostics settings reported in the main results.

Split. The explicit split table below documents the DS02 protocol because it is the N-CMAPSS split currently tabulated in the manuscript. DS02 is split into five training units, one validation unit, and three test units, with the split shown in Table 7. The broader N-CMAPSS settings remain part of the same source family and are reported separately in the main results as NC-P and NC-D.

Target, normalization, and metrics. For NC-DS02 and NC-P, the target is RUL, measured as the remaining flights or cycles before end-of-life. Features are min-max normalized, and prognostics metrics are MAE, MSE, and RMSE of estimated RUL against the true value. For NC-D, the target is a diagnostic class label and the main-results metric is F1 score.

A.4. Hydraulic Diagnostics (HSF15)

HSF15. Description. HSF15 [25] is a hydraulic-system condition-monitoring benchmark based on a laboratory test rig with multivariate sensor measurements. This work evaluates it as four component-level diagnostics tasks: accumulator (HSF15-A), cooler (HSF15-C), pump (HSF15-P), and valve (HSF15-V).

Split. All four HSF15 tasks use the same datasource family and split protocol, but differ in the target component and number of fault classes. The tasks are reported separately in the main benchmark table because each component defines a distinct diagnostic classification problem.

Target, normalization, and metrics. The target is the component-specific fault class. Features are scaled according to the shared preprocessing pipeline, and the tabular fit-predict pipeline summarizes sensor bursts through time-domain and spectral descriptors before tabularization. The main-results metric is F1 score.

A.5. PHME20

PHME20. Description. PHME20 [29] is the PHM Society 2020 European Conference Data Challenge dataset. It records an experimental industrial filtration system in which a particulate-laden gas stream progressively clogs a filter, increasing differential pressure until an operational threshold is reached. Each run captures one filter lifetime under varying dust and feed conditions.

Split. This work follows the challenge-provided split, which assigns disjoint filter runs to training, validation, and test partitions. No additional dataset-specific filtering is applied in this protocol.

Target, normalization, and metrics. PHME20 is used as a prognostics task with a direct per-timestep RUL target. The preprocessing pipeline min-max scales the sensor channels and RUL target. The task logs MAE, MSE, RMSE, and `nasa_score`.

Fault Category	Fault days
Outdoor Air Damper Stuck	5
Heating Coil Valve Leaking	3
Cooling Coil Valve Leaking	5
Unfaulted	13

Table 8

Number of daulty days per fault group

A.6. MZVAV

MZVAV. Description. MZVAV is an automated fault detection and diagnostics dataset stemming from a small commerical building with multi-zone variable air volume system generated by Drexel university in the ASHRAE 1312 project. [21]. It is a simulated building-fault dataset featuring three air-handling units with 18 sensors collected during 26 days across summer, winter, and transition seasons with a one-minute resolution. The operation faults were manually imposed into the control system. The collected signals include outdoor air temperature, supply air temperature and set-point, mixed air and return air temperature, supply air fan status and speed control, return air fan status and speed control, exhaust air damper control, outdoor air and return air damper control, cooling and heating coil valve control, supply air duct static pressure and set point, occupancy mode indicator and fault detection ground truth. On this basis, we formulate a multi-class classification problem with the fault detection ground truth as label.

Split. Individual faults are grouped into the 4 classes "Outdoor Air Damper Stuck", "Heating Coil Valve Leaking", "Cooling Coil Valve Leaking", and "Unfaulted". The number of days per group is shown in Table 8. The benchmark uses a stratified train/validation/test split over days with a test size of 20%.

Target, normalization, and metrics. The target is the fault detection ground truth. Features are min-max scaled to [0, 1]. Metrics are F1-score, precision, recall, and accuracy.

B. Implementation and Model Details

B.1. Sequential Models and In-Context Tabular Models

This subsection collects the implementation details needed to connect the protocol above to the evaluated model families. The key distinction is the input representation: sequence models consume $\mathcal{D}_{\text{seq}}$, while tabular foundation models and tabular baselines consume $\mathcal{D}_{\text{tab}}$.

Conventional sequence models, such as 1D-CNNs, LSTMs, and Time-Series Transformers, are trained to minimize a task-specific loss over the training sequence dataset $\mathcal{D}_{\text{seq}}^{\text{train}}$. These models do not operate on tabularized rows $\mathbf{X}_m$; instead, they ingest the temporal window $\mathbf{W}_m$ directly.

Let f_θ denote a model parameterized by θ . The model produces a prediction $\hat{y}_m = f_\theta(\mathbf{W}_m)$. The optimal parameters θ^* are obtained by minimizing empirical risk over the training sequence dataset:

$$\theta^* = \arg \min_{\theta} \frac{1}{|\mathcal{D}_{\text{seq}}^{\text{train}}|} \sum_{(\mathbf{W}_m, y_m) \in \mathcal{D}_{\text{seq}}^{\text{train}}} \ell(f_\theta(\mathbf{W}_m), y_m) \quad (29)$$

where $\ell(\cdot, \cdot)$ is a task-dependent loss function, such as mean squared error for prognostics or cross-entropy for diagnostics. We typically employ stochastic gradient descent variants such as AdamW to solve this optimization problem.

Tabular foundation models operate through in-context learning. In this paradigm, the model f_ϕ , parameterized by weights ϕ , predicts for a query row $\mathbf{X}_q$ by conditioning on a context set $\mathcal{C}$ of labeled support examples:

$$\hat{y}_q = f_\phi(\mathcal{C}, \mathbf{X}_q) \quad (30)$$

To ensure a valid evaluation and prevent leakage, the context set must be drawn strictly from the training partition of the tabular dataset:

$$\mathcal{C} = \{(\mathbf{X}_j, y_j)\}_{j=1}^C \subset \mathcal{D}_{\text{tab}}^{\text{train}}. \quad (31)$$

XGBoost is included as a classical tree-based tabular baseline [10]. It uses the same tabularized rows $\mathbf{X}_m$ as the tabular foundation models, but learns task-specific boosted trees from the training partition rather than using in-context prediction.

B.2. Evaluation Protocol and Baseline Adaptation

All models are evaluated on the same held-out instances derived from $\mathcal{D}^{\text{test}}$. The only difference is the representation supplied to the model: sequence models consume $\mathbf{W}_m$, while tabular models consume $\mathbf{X}_m$ and, when applicable, a training-only context set $\mathcal{C}$.

For a task-specific loss $\ell(\hat{y}, y)$, the aggregate test loss is computed over the same test index set for both model families:

$$\mathcal{L}_{\text{test}} = \frac{1}{|\mathcal{D}^{\text{test}}|} \sum_{m \in \mathcal{D}^{\text{test}}} \ell(\hat{y}_m, y_m), \quad \text{where } \hat{y}_m = \begin{cases} f_{\theta}(\mathbf{W}_m) & \text{if } f \text{ is a sequence model,} \\ f_{\phi}(\mathcal{C}, \mathbf{X}_m) & \text{if } f \text{ is an in-context tabular model.} \end{cases} \quad (32)$$

For prognostics, this work reports regression metrics such as RMSE and MAE. For diagnostics, it reports classification metrics such as accuracy and macro-F1. For cross-dataset comparisons, models are ranked within each dataset and the average rank is reported, with rank 1 denoting the best model for a given dataset.

Baseline models trained from scratch, namely Bi-LSTM, 1D-CNN, TiDE, and Transformer-based models, are applied directly to the sequence windows $\mathbf{W}_m$. Some Transformer models, including the Time-Series Transformer, Crossformer, and Spacetimeformer, use an encoder-decoder formulation in their original forecasting setting. That formulation assumes historical target values, historical covariates, and future-horizon covariates.

This formulation is not directly applicable to PHM prognostics and diagnostics because the past values of the target variable, such as remaining useful life, are unavailable at inference time, and no future information is used. We therefore adapt encoder-decoder Transformer architectures by removing the encoder and directly feeding $\mathbf{W}_m$ to the decoder. This step is not needed for PatchTST, which is decoder-only by design. For PatchTST, channel-independent processing cannot be applied to the regression target for the same reason, so decoder latents are concatenated and a separate regression or classification head is trained. For TiDE, we disable the encoder pathway and use the dynamic covariate pathway to process $\mathbf{W}_m$. For diagnostics, the regression head is replaced with a classification head that outputs n classes.

The transformer training regimen uses a warm-up phase and reduces the learning rate upon plateau. Min-max or z-score normalization is selected depending on the dataset. All preprocessing, feature extraction, and sequence slicing, represented by $\mathcal{G}$ and $\mathcal{S}$, are shared among the models; tabular foundation models and XGBoost additionally share the tabularization schema $\mathcal{T}$.

B.3. TabPFN and TabDPT Details

TabPFN [26, 27] is a Prior-Data Fitted Network (PFN) trained on a large-scale synthetic dataset that consists of millions of small tabular instances. These tabular instances are generated from randomly sampled structural causal models. TabPFN therefore leverages the structural diversity seen during pre-training to implicitly understand and generalize to the relational structure of unseen tabular data. This enables the network to efficiently approximate the Bayesian posterior for the chosen prior:

$$p(y^{\text{test}} | \mathcal{D}_{\text{tab}}^{\text{test}}, \mathcal{Y}_{\text{tab}}^{\text{train}}, \mathcal{D}_{\text{tab}}^{\text{train}}) \quad (33)$$

Internally, TabPFN converts tabular cells into a sequence of high-dimensional feature tokens by mapping every categorical or scalar attribute to a fixed-dimensional embedding. The attribute embedding strategy is therefore similar to Spacetimeformer, which flattens spatial-temporal time sequences and embeds each signal and time coordinate in $\mathbf{W}_k \in \mathbb{R}^{L_{\text{seq}} \times F}$ for a total of $L_{\text{seq}} \times F$ embeddings. In TabPFN, the final embedding is obtained by multiplying each raw feature value with an attribute-specific offset vector that places all attributes in a shared latent space. TabPFN is a univariate model. For multivariate outputs in PHM, we therefore parallelize the process by fitting one model per variate. To further enhance robustness, we use an ensemble of eight TabPFN estimators.

We also evaluate TabDPT [38], a tabular foundation model that has been trained purely on real data. Due to its capability to perform classification and regression, we employ TabDPT as a retrieval-based discriminatively pretrained transformer that operates on full row representations. TabDPT supports up to $D_{\text{max}} = 100$ features, so when D_{tab} exceeds this limit TabDPT applies Principal Component Analysis to obtain a projection with exactly D_{max} dimensions.

TabDPT separately embeds features and labels linearly into a 768 dimensional space and combines them through element-wise addition.

During inference, TabDPT builds a local context by selecting the top K most similar training rows using the FAISS library [15]. The full transformer sequence is formed by concatenating the embedded training samples and the embedded evaluation sample, where the training embeddings are shifted by their label embeddings and the evaluation sample is kept unlabeled. As for TabPFN, we use the ensemble of eight TabDPT estimators, each trained on a random subset of columns, and aggregate their outputs through a weighted average. We ensure that there is no overlap between the datasets evaluated in this work and the pre-training data of TabPFN.

C. Additional Experimental Results

This appendix presents the complete experimental setup and results for the benchmark evaluation. It documents the evaluated experiment families, preprocessing schemas, and hyperparameter search spaces. For the condensed main-text summary, see Section 6; for dataset-level protocol details, see Appendix A.

C.1. Experimental setup

Experiment definitions. Each experiment family is defined by a Hydra configuration [60] that specifies the datasource, transform pipeline, model, evaluator, seed set, and hyperparameter search space. The benchmark covers gradient-trained and fit-predict models for both diagnostics and prognostics tasks. XGBoost uses a separate fit-predict configuration that reuses the same experiment definitions and transform families as the other tabular models.

Learning tasks and datasets. The evaluation covers two PHM task categories. Diagnostics comprises multiclass fault classification on MZVAV [22], four component-specific hydraulic diagnostics tasks on HSF15 [25] (accumulator, cooler, pump, and valve), and concept classification on N-CMAPSS Multi [4, 20]. Prognostics comprises ah-RUL regression on NB14 [5] and UNIBO21 [58], direct RUL regression on PHME20 [29], RUL prediction on the N-CMAPSS Multi and DS02 families, and bearing prognostics on XJTU-SY [61].

Target semantics differ by family. NB14 and UNIBO21 predict ah-RUL, i.e., remaining cumulative discharge throughput in Ampere-hours; PHME20 and the N-CMAPSS families predict remaining useful life; and the bearing family transforms runtime trajectories into a normalized degradation target through the configured health-index transform, while evaluation is reported with the per-unit metric suite. All families use disjoint train/validation/test entities according to their datasource definitions, with MZVAV as the only day-stratified diagnostics exception.

Models. The benchmark compares five model families: (i) simple baselines (Linear, Exp, MLP), (ii) deep sequence models (LSTM, CNN-1D, TiDE), (iii) transformers (TST, STF, CF, PTST), (iv) tabular models (XGBoost), and (v) tabular foundation models (TabPFN, TabDPT). Linear is regression-only and is therefore omitted from diagnostics; for diagnostics, a linear classifier serves the analogous baseline role.

Training and evaluation. Every model–dataset configuration is repeated over five random seeds. Gradient-trained models search over `seq_len` $\in \{1, 10, 50\}$ and `lr` $\in \{0.001, 0.0005, 0.0001\}$, with batch size 512, a maximum of 200 epochs, and early stopping. Fit-predict models search over context/stride pairs (1, 1), (5, 1), (10, 5), (20, 5), and (50, 50). Diagnostics selects the best configuration on `val/f1`; prognostics (generic RUL objective) selects on `val/loss`. Evaluation uses three evaluator families: `classification` (diagnostics), `rul` (direct RUL regression), and `per_unit` (battery and bearing families).

C.2. Transformation schemas

All experiment families obey the same leakage-prevention invariant: any fitted normalization or feature-extraction statistic is estimated on the training partition only and then reused unchanged for validation and test. The subsections below describe the preprocessing pipeline applied to each dataset group; Across all families, the transformed tensors are subsequently windowed according to the sequence-length and stride settings described in Section C.3.

C.2.1. Battery datasets (NB14 and UNIBO21)

For both NB14 and UNIBO21, raw sensor channels and the ah-RUL target are min-max scaled (MinMaxScaler). Two descriptor branches are then extracted from the cycle traces: time-domain statistics—mean, variance, kurtosis, peak factor, and related summaries (TimeStatsTransform)—and frequency-domain statistics via FFT (SpectralStatsTransform). The resulting descriptor vectors are concatenated (ConcatenateTransform) and re-scaled with a final min-max transform. For fit-predict models, the pipeline additionally tabularizes the processed history into a single feature vector for one-shot inference (TimeseriesTabularizer). This pipeline corresponds to the `combined / combined_fit_predict` configuration group.

C.2.2. Bearing dataset (XJTU-SY)

For XJTU-SY, raw vibration channels are standardized with training-partition statistics (StandardScaler). Runtime targets are converted into a normalized degradation trajectory via HealthIndexTransform. An additional cumulative-sum feature is computed and min-max scaled, and time-domain plus spectral descriptors (TimeStatsTransform,

Table 9

Hyperparameter search families used in the benchmark evaluation.

Search family	Models	Search space	Fixed settings	Selection rule
Gradient-trained	LSTM, 1D-CNN, STF, Cross-former, Timeseries Transformer, TiDE, PatchTST, MLP, linear classifier, linear regression, exponential regression	$\text{seq_len} \in \{1, 10, 50\}$; $\in \{0.001, 0.0005, 0.0001\}$	lr Batch size 512; max 200 epochs; early stopping	val/f1 for diagnostics; val/loss for prognostics
Fit-predict	XGBoost, TabPFN, TabDPT	Context/stride pairs (1, 1), (5, 1), (10, 5), (20, 5), (50, 50)	Five seeds; deterministic transform reuse; one-shot fit/predict evaluation	val/f1 for diagnostics; val/loss for prognostics

SpectralStatsTransform) are extracted from the standardized vibration signals. The resulting features are concatenated and min-max rescaled. For fit-predict models, the pipeline tabularizes the combined representation while preserving unit identifiers for per-unit evaluation. This pipeline corresponds to the `combined / combined_fit_predict` configuration group.

C.2.3. *N-CMAPSS families*

Sensor features and operating descriptors are standardized with fixed N-CMAPSS scalers (N_CMAPSSDescriptorsScaler, StandardScaler), and the RUL label is multiplied by the constant factor 0.01 (ConstantScaler). Each flight is then temporally aggregated with non-overlapping windows of 60 timesteps (WindowedAggregationTransform), and the aggregated sensor features and descriptors are concatenated into a single input representation. For the multi-source diagnostics family, the transform additionally builds unified concept classes from the source-specific concept annotations. Fit-predict models tabularize the aggregated histories after the same scaling and concatenation stages. This pipeline corresponds to the `depater2023_default / depater2023_fit_predict_history` configuration group.

C.2.4. *Building diagnostics (MZVAV)*

No additional feature engineering is applied for MZVAV. The pipeline rescales the sensor channels with a precomputed dataset-specific min-max scaler (MinMaxScalerMZVAV) and routes the scalar target directly to the fault-classification key consumed by the diagnostics models. For fit-predict models, the pipeline tabularizes the resulting history windows without introducing a separate descriptor stage. This pipeline corresponds to the `default / fit_predict_history` configuration group.

C.2.5. *Hydraulic diagnostics (HSF15)*

All four HSF15 component tasks share the same preprocessing pipeline. Features are min-max scaled (MinMaxScaler), the component-specific target is reduced to the last label in each window and assigned to the fault-classification key, and the sensor burst is summarized through time-domain statistics (TimeStatsTransform) and spectral statistics (SpectralStatsTransform), followed by concatenation and a final min-max rescaling. For fit-predict models, the pipeline tabularizes the statistics-based representation for XGBoost, TabPFN, and TabDPT. This pipeline corresponds to the `default / statistics_fit_predict` configuration group.

C.2.6. *PHM challenge prognostics (PHME20)*

Both features and the direct RUL target are min-max scaled (MinMaxScaler), with no additional handcrafted feature extraction. For fit-predict models, the pipeline adds history tabularization for one-shot inference. This pipeline corresponds to the `normalize_feature_target / normalize_feature_target_fit_predict` configuration group.

C.3. Hyperparameter search

The benchmark exposes two hyperparameter-search families: a gradient-trained family for sequence models and simple neural baselines, and a fit-predict family for tabular/foundation models including XGBoost. Table 9 summarizes the search spaces.

Abbreviation	Task	Source dataset (domain)
NC-DS02	Prognostics	N-CMAPSS DS02 (turbofan engine RUL)
NC-P	Prognostics	N-CMAPSS Multi-source (turbofan engine RUL)
NB14	Prognostics	NASA Randomized Battery Usage (battery ah-RUL)
PHME20	Prognostics	PHM 2020 Challenge (industrial filtration RUL)
Unibo	Prognostics	UNIBO Powertools (battery ah-RUL)
XJTU-SY	Prognostics	XJTU-SY (bearing degradation)
NC-D	Diagnostics	N-CMAPSS Multi-source (turbofan concept classification)
HSF15-A	Diagnostics	HSF15 (hydraulic accumulator, 4-way)
HSF15-C	Diagnostics	HSF15 (hydraulic cooler, 3-way)
HSF15-P	Diagnostics	HSF15 (hydraulic pump, 3-way)
HSF15-V	Diagnostics	HSF15 (hydraulic valve, 4-way)
MZVAV	Diagnostics	MZVAV (multi-zone HVAC fault, 4-way)

All successful runs persist the resolved configuration, Hydra override trace, run metadata, and reproduction instructions alongside the predictions and evaluator outputs. The corresponding artifact protocol is described in Appendix D.

C.4. Reading the result tables

The result tables in Sections C.5–C.6 share a common reading convention, summarized here once so individual captions can stay terse.

Dataset abbreviations. Column headers use short identifiers consistently across both the main-text and appendix tables. Their full meanings are:

Cell convention. Each cell reports mean $\pm$ std over five independent seeds. Bold cells are the best in their column; underlined cells are the second best. Rows are grouped by model family in the order: simple baselines, deep sequence models, transformers, tabular models, tabular foundation models.

Metric scaling and direction. Diagnostics metrics (F1, AUROC, Accuracy) are reported on a 0–100 scale; prognostics MAE/MSE are reported both in the framework’s normalized target space (reported $\times 100$) and in original engineering units (denormalized) for practitioner interpretation. Arrows in column headers indicate metric direction ($\downarrow$ lower is better, $\uparrow$ higher is better). The *Avg rank* column is the mean of per-task ranks across the columns of that table.

C.5. Diagnostics results

Diagnostics is evaluated by F1 (the headline metric in Section 6) and complemented here by AUROC and Accuracy. The three metrics agree on the top group: TabDPT, TabPFN, CNN-1D, and XGBoost cluster within 1.0 average-rank of each other on F1, with LSTM joining the leading tier on F1 and Accuracy. MZVAV (multi-zone HVAC fault classification under a day-stratified split) is the hardest task in every metric and the only family where the gap to chance is small. The metric-robustness check therefore supports the F1 choice in the main text.

Cross-family observations. Across the three diagnostics metrics, the model families separate clearly. Tabular foundation models (TabDPT, TabPFN) and XGBoost rank in the top tier even though they consume tabularized windows rather than raw sequences; this is most visible on HSF15 (where many models reach 100 F1), but it also holds on harder tasks such as NC-D and MZVAV. Deep sequence models split into a strong pair (LSTM, CNN-1D) and a weaker TiDE variant under this training budget. By contrast, the transformer family (TST, STF, CF, PTST) performs near chance on most diagnostics tasks, despite consuming identical inputs under the same protocol. Simple baselines (MLP, Linear) remain competitive on several HSF15 components, reinforcing that hydraulic component diagnostics is comparatively easy relative to MZVAV and NC-D. Finally, the robustness check is consistent: F1, AUROC, and Accuracy induce nearly identical rankings, so the headline conclusions do not depend on the specific metric choice.

Table 10F1 score on diagnostics ($\uparrow$). Same numbers as the diagnostics floor of Table 2, reproduced here with full per-task breakdown.

Model	NC-D	HSF15-A	HSF15-C	HSF15-P	HSF15-V	MZVAV	Average rank
Linear	72.34 $\pm$ 3.04	58.75 $\pm$ 1.81	98.35 $\pm$ 0.82	54.40 $\pm$ 11.97	32.55 $\pm$ 2.36	39.89 $\pm$ 8.77	7.33
MLP	79.49 $\pm$ 1.80	91.02 $\pm$ 2.25	99.91 $\pm$ 0.14	97.32 $\pm$ 0.60	80.99 $\pm$ 29.38	60.10 $\pm$ 6.39	5.00
LSTM	88.84 $\pm$ 0.73	94.59 $\pm$ 0.97	100.00 $\pm$ 0.00	95.94 $\pm$ 2.83	97.35 $\pm$ 3.32	51.31 $\pm$ 6.01	3.83
CNN-1D	87.53 $\pm$ 2.83	94.03 $\pm$ 1.98	100.00 $\pm$ 0.00	98.73 $\pm$ 0.52	97.92 $\pm$ 0.83	66.11 $\pm$ 5.74	3.00
TiDE	32.57 $\pm$ 4.65	42.90 $\pm$ 5.07	61.57 $\pm$ 10.41	59.37 $\pm$ 7.68	42.11 $\pm$ 14.36	25.19 $\pm$ 5.29	8.17
TST	26.34 $\pm$ 3.96	37.37 $\pm$ 4.38	59.18 $\pm$ 10.29	46.07 $\pm$ 4.13	35.40 $\pm$ 5.77	24.93 $\pm$ 4.38	10.00
STF	24.55 $\pm$ 3.60	40.01 $\pm$ 5.75	65.94 $\pm$ 20.61	50.39 $\pm$ 11.54	37.34 $\pm$ 5.68	38.01 $\pm$ 5.56	8.67
CF	23.74 $\pm$ 0.93	25.04 $\pm$ 5.28	59.19 $\pm$ 10.05	29.46 $\pm$ 2.61	23.98 $\pm$ 2.95	17.34 $\pm$ 4.24	11.50
PTST	19.57 $\pm$ 0.26	31.56 $\pm$ 4.04	41.57 $\pm$ 5.18	41.22 $\pm$ 6.63	26.20 $\pm$ 2.44	25.81 $\pm$ 4.15	11.00
XGBoost	48.13 $\pm$ 0.00	98.07 $\pm$ 0.00	100.00 $\pm$ 0.00	99.66 $\pm$ 0.00	99.65 $\pm$ 0.00	57.08 $\pm$ 0.00	3.17
TabPFN	67.15 $\pm$ 1.46	99.47 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	58.32 $\pm$ 2.44	2.33
TabDPT	85.21 $\pm$ 0.16	96.66 $\pm$ 1.03	100.00 $\pm$ 0.00	99.06 $\pm$ 0.25	98.92 $\pm$ 0.35	71.29 $\pm$ 0.48	2.33

Table 11AUROC on diagnostics ($\uparrow$).

Model	NC-D	HSF15-A	HSF15-C	HSF15-P	HSF15-V	MZVAV	Average rank
Linear	93.76 $\pm$ 1.29	85.10 $\pm$ 1.35	99.43 $\pm$ 0.63	77.41 $\pm$ 12.99	64.74 $\pm$ 4.49	71.47 $\pm$ 9.58	7.17
MLP	96.31 $\pm$ 0.67	98.95 $\pm$ 0.39	100.00 $\pm$ 0.00	99.84 $\pm$ 0.07	90.02 $\pm$ 21.11	85.24 $\pm$ 2.03	5.17
LSTM	98.41 $\pm$ 0.16	99.54 $\pm$ 0.11	100.00 $\pm$ 0.00	99.68 $\pm$ 0.27	99.88 $\pm$ 0.17	86.32 $\pm$ 4.06	3.33
CNN-1D	98.11 $\pm$ 0.62	99.44 $\pm$ 0.19	100.00 $\pm$ 0.00	99.94 $\pm$ 0.06	99.94 $\pm$ 0.05	85.45 $\pm$ 2.29	3.17
TiDE	62.71 $\pm$ 3.84	69.00 $\pm$ 5.66	81.46 $\pm$ 5.14	78.78 $\pm$ 11.24	64.91 $\pm$ 11.98	60.77 $\pm$ 9.35	8.00
TST	57.03 $\pm$ 2.91	62.99 $\pm$ 6.37	79.87 $\pm$ 9.04	65.30 $\pm$ 6.07	61.60 $\pm$ 5.19	57.44 $\pm$ 9.01	9.33
STF	55.23 $\pm$ 3.04	58.44 $\pm$ 5.10	81.96 $\pm$ 14.13	64.97 $\pm$ 9.06	59.98 $\pm$ 15.81	62.61 $\pm$ 10.35	9.50
CF	52.00 $\pm$ 0.87	53.48 $\pm$ 5.54	73.59 $\pm$ 8.17	37.35 $\pm$ 0.53	51.76 $\pm$ 0.94	47.64 $\pm$ 6.33	11.67
PTST	50.01 $\pm$ 0.24	58.74 $\pm$ 1.64	59.83 $\pm$ 4.32	62.24 $\pm$ 5.03	52.84 $\pm$ 2.86	52.97 $\pm$ 5.01	11.17
XGBoost	82.56 $\pm$ 0.00	99.94 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	84.51 $\pm$ 0.00	3.17
TabPFN	93.97 $\pm$ 0.31	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	83.67 $\pm$ 0.77	2.67
TabDPT	97.38 $\pm$ 0.06	99.91 $\pm$ 0.05	100.00 $\pm$ 0.00	99.99 $\pm$ 0.00	99.99 $\pm$ 0.00	88.57 $\pm$ 0.73	2.50

Table 12Accuracy on diagnostics ($\uparrow$).

Model	NC-D	HSF15-A	HSF15-C	HSF15-P	HSF15-V	MZVAV	Average rank
Linear	73.79 $\pm$ 2.35	61.47 $\pm$ 1.91	98.36 $\pm$ 0.82	63.08 $\pm$ 14.59	45.93 $\pm$ 2.01	61.56 $\pm$ 4.94	7.33
MLP	79.72 $\pm$ 1.82	92.39 $\pm$ 2.06	99.91 $\pm$ 0.14	98.05 $\pm$ 0.41	85.28 $\pm$ 22.19	71.54 $\pm$ 4.90	5.00
LSTM	88.42 $\pm$ 0.72	95.42 $\pm$ 0.82	100.00 $\pm$ 0.00	96.96 $\pm$ 1.94	97.64 $\pm$ 3.10	70.71 $\pm$ 4.77	3.50
CNN-1D	86.91 $\pm$ 2.91	95.00 $\pm$ 1.57	100.00 $\pm$ 0.00	99.09 $\pm$ 0.36	98.38 $\pm$ 0.67	77.81 $\pm$ 3.10	3.00
TiDE	35.30 $\pm$ 3.73	49.24 $\pm$ 10.75	65.39 $\pm$ 8.98	70.93 $\pm$ 3.78	57.08 $\pm$ 10.30	30.15 $\pm$ 7.47	8.50
TST	26.80 $\pm$ 3.37	43.36 $\pm$ 8.23	62.53 $\pm$ 10.12	52.24 $\pm$ 6.62	45.19 $\pm$ 6.74	45.19 $\pm$ 9.25	9.83
STF	24.82 $\pm$ 3.12	47.02 $\pm$ 9.33	72.30 $\pm$ 15.62	64.90 $\pm$ 9.85	57.59 $\pm$ 4.35	50.34 $\pm$ 9.76	8.50
CF	28.90 $\pm$ 3.18	30.71 $\pm$ 5.80	61.83 $\pm$ 10.46	35.15 $\pm$ 4.49	33.06 $\pm$ 3.25	27.02 $\pm$ 10.45	11.17
PTST	21.56 $\pm$ 0.93	34.41 $\pm$ 4.03	42.44 $\pm$ 4.66	43.36 $\pm$ 7.01	31.85 $\pm$ 5.87	39.70 $\pm$ 5.78	11.33
XGBoost	54.75 $\pm$ 0.00	98.53 $\pm$ 0.00	100.00 $\pm$ 0.00	99.77 $\pm$ 0.00	99.77 $\pm$ 0.00	62.62 $\pm$ 0.00	3.33
TabPFN	70.61 $\pm$ 1.07	99.58 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	65.55 $\pm$ 1.96	2.50
TabDPT	84.74 $\pm$ 0.16	97.27 $\pm$ 0.83	100.00 $\pm$ 0.00	99.27 $\pm$ 0.19	99.17 $\pm$ 0.26	80.10 $\pm$ 0.36	2.33

C.6. Prognostics results

The prognostics metric surface is reported in three families. Aggregate MAE and MSE (normalized and denormalized) are computed by pooling all predictions and all units before averaging. Per-unit aggregation, defined only on battery and bearing tasks where the framework’s per_unit evaluator is active, computes one error per monitored unit before averaging across units; this tightens rankings among leading models and reduces the influence of long-trajectory

Table 13

MAE on prognostics. Top block: normalized target space ($\times 100$) ($\downarrow$); bottom block: original engineering units ($\downarrow$). Same numbers as the prognostics floor of Table 2 (top), reproduced with full per-task breakdown.

Model	NC-DS02	NC-P	NB14	PHME20	Unibo	XJTU-SY	Average rank
<i>Normalized target space ($\downarrow$)</i>							
Linear	10.13 $\pm$ 0.14	16.11 $\pm$ 0.60	41.69 $\pm$ 12.02	12.19 $\pm$ 0.36	27.59 $\pm$ 14.36	76.80 $\pm$ 60.41	12.50
Exp	5.35 $\pm$ 0.06	10.96 $\pm$ 0.09	30.47 $\pm$ 47.76	8.82 $\pm$ 0.52	12.19 $\pm$ 0.31	27.22 $\pm$ 4.06	9.67
MLP	6.37 $\pm$ 0.23	13.17 $\pm$ 0.78	14.38 $\pm$ 9.77	4.62 $\pm$ 1.15	12.50 $\pm$ 0.76	30.64 $\pm$ 2.67	10.33
LSTM	4.93 $\pm$ 0.13	7.56 $\pm$ 0.31	3.80 $\pm$ 0.22	3.73 $\pm$ 0.98	6.50 $\pm$ 0.16	21.89 $\pm$ 0.40	3.67
CNN-1D	5.33 $\pm$ 0.37	7.53 $\pm$ 0.22	8.89 $\pm$ 1.70	5.35 $\pm$ 3.71	12.41 $\pm$ 1.15	31.02 $\pm$ 8.25	8.67
TiDE	5.29 $\pm$ 0.22	7.62 $\pm$ 0.20	3.44 $\pm$ 0.17	4.20 $\pm$ 0.66	6.46 $\pm$ 0.78	25.11 $\pm$ 2.38	5.17
TST	5.31 $\pm$ 0.13	7.02 $\pm$ 0.17	6.28 $\pm$ 0.25	4.11 $\pm$ 0.84	7.23 $\pm$ 0.39	33.30 $\pm$ 7.72	7.00
STF	4.89 $\pm$ 0.10	7.35 $\pm$ 1.16	10.67 $\pm$ 3.16	3.91 $\pm$ 1.00	8.89 $\pm$ 0.81	28.49 $\pm$ 4.01	6.17
CF	5.76 $\pm$ 0.51	9.98 $\pm$ 0.57	3.57 $\pm$ 0.07	3.87 $\pm$ 0.85	5.58 $\pm$ 1.08	22.09 $\pm$ 1.06	5.00
PTST	16.62 $\pm$ 0.04	21.55 $\pm$ 0.03	5.22 $\pm$ 0.10	15.09 $\pm$ 1.13	11.18 $\pm$ 1.11	25.42 $\pm$ 1.48	10.33
XGBoost	8.52 $\pm$ 0.00	15.24 $\pm$ 0.00	4.48 $\pm$ 0.00	2.68 $\pm$ 0.00	4.06 $\pm$ 0.00	24.59 $\pm$ 0.00	6.50
TabPFN	4.96 $\pm$ 0.04	7.79 $\pm$ 0.04	3.91 $\pm$ 0.03	1.95 $\pm$ 0.03	3.72 $\pm$ 0.06	22.27 $\pm$ 0.35	3.33
TabDPT	5.07 $\pm$ 0.06	6.85 $\pm$ 0.02	3.63 $\pm$ 0.04	2.19 $\pm$ 0.01	3.94 $\pm$ 0.05	23.24 $\pm$ 0.45	2.67
<i>Original engineering units ($\downarrow$)</i>							
Linear	10.13 $\pm$ 0.14	16.11 $\pm$ 0.60	451.64 $\pm$ 130.15	43.13 $\pm$ 1.27	135.80 $\pm$ 70.68	907.40 $\pm$ 774.36	12.50
Exp	5.35 $\pm$ 0.06	10.96 $\pm$ 0.09	330.02 $\pm$ 517.36	31.21 $\pm$ 1.85	60.02 $\pm$ 1.51	349.71 $\pm$ 60.98	9.67
MLP	6.37 $\pm$ 0.23	13.17 $\pm$ 0.78	155.77 $\pm$ 105.84	16.35 $\pm$ 4.06	61.51 $\pm$ 3.72	384.95 $\pm$ 40.56	10.50
LSTM	4.93 $\pm$ 0.13	7.56 $\pm$ 0.31	41.13 $\pm$ 2.34	13.19 $\pm$ 3.47	32.00 $\pm$ 0.77	271.33 $\pm$ 3.45	3.67
CNN-1D	5.33 $\pm$ 0.37	7.53 $\pm$ 0.22	96.25 $\pm$ 18.38	18.95 $\pm$ 13.12	61.10 $\pm$ 5.66	380.05 $\pm$ 96.92	8.50
TiDE	5.29 $\pm$ 0.22	7.62 $\pm$ 0.20	37.22 $\pm$ 1.80	14.86 $\pm$ 2.35	31.78 $\pm$ 3.86	320.41 $\pm$ 24.80	5.33
TST	5.31 $\pm$ 0.13	7.02 $\pm$ 0.17	68.00 $\pm$ 2.71	14.56 $\pm$ 2.96	35.60 $\pm$ 1.94	424.88 $\pm$ 100.25	7.00
STF	4.89 $\pm$ 0.10	7.35 $\pm$ 1.16	115.61 $\pm$ 34.23	13.83 $\pm$ 3.53	43.76 $\pm$ 4.01	361.41 $\pm$ 54.88	6.17
CF	5.76 $\pm$ 0.51	9.98 $\pm$ 0.57	38.70 $\pm$ 0.80	13.71 $\pm$ 3.02	27.48 $\pm$ 5.31	282.20 $\pm$ 19.10	5.00
PTST	16.62 $\pm$ 0.04	21.55 $\pm$ 0.03	56.53 $\pm$ 1.09	53.40 $\pm$ 3.99	55.02 $\pm$ 5.47	312.68 $\pm$ 23.46	10.17
XGBoost	8.52 $\pm$ 0.00	15.24 $\pm$ 0.00	48.54 $\pm$ 0.00	9.48 $\pm$ 0.00	19.98 $\pm$ 0.00	311.19 $\pm$ 0.00	6.50
TabPFN	4.96 $\pm$ 0.04	7.79 $\pm$ 0.04	42.32 $\pm$ 0.28	6.89 $\pm$ 0.09	18.33 $\pm$ 0.30	292.43 $\pm$ 5.26	3.50
TabDPT	5.07 $\pm$ 0.06	6.85 $\pm$ 0.02	39.35 $\pm$ 0.47	7.75 $\pm$ 0.04	19.42 $\pm$ 0.24	282.83 $\pm$ 5.84	2.50

units that dominate window-level pools. Domain-specific scores (NASA, PHM) are reported with the per-task scoping enforced by the framework’s metric registry.

C.6.1. Aggregate errors

Tables 13 and 14 report MAE and MSE in a two-block format. The top block reports errors in the normalized target space (reported $\times 100$), which is used to compute cross-task *Avg rank*; the bottom block reports the same predictions in original engineering units for practitioner interpretation. MSE emphasizes occasional large errors, but it preserves the headline ordering among the leading models. Because MSE penalizes large residuals quadratically, it is sensitive to rare catastrophic predictions; we report it alongside MAE to make these failure modes visible.

Cross-family observations. Prognostics rankings are more compressed than diagnostics. TabDPT, TabPFN, and LSTM lead on combined rank, but CF, TiDE, and STF remain within a few rank points and each takes at least one column-best on normalized MAE. Two patterns stand out. First, the transformer family that collapses on diagnostics is competitive on prognostics (e.g., STF is best on NC-DS02 and TST is near the top on NC-P), suggesting a task-specific failure mode rather than an architecture-wide limitation. Second, simple baselines (Linear, Exp) degrade more on prognostics than on diagnostics: Exp is consistently far from the leaders, while Linear remains mid-tier. Switching from MAE to MSE tightens the leading group and penalizes rare catastrophic errors, but it does not change the top of the leaderboard.

C.6.2. Per-unit aggregation (battery and bearing families)

Battery (NB14, UNIBO21) and bearing (XJTU-SY) prognostics are evaluated trajectory-level rather than window-level: the framework’s *per_unit* evaluator computes one error per monitored unit and then aggregates. Table 15 reports the per-unit-mean MAE on these three families in the same two-floor form (normalized top, denormalized bottom). The values are produced from the same predictions as the aggregate tables above; the difference is purely

Table 14

MSE on prognostics. Top block: normalized target space ($\times 100$) ($\downarrow$); bottom block: original engineering units ($\downarrow$). MSE redistributes weight onto large per-window errors but preserves the leading-model ranking from MAE.

Model	NC-DS02	NC-P	NB14	PHME20	Unibo	XJTU-SY	Average rank
<i>Normalized target space ($\downarrow$)</i>							
Linear	1.37 $\pm$ 0.04	3.95 $\pm$ 0.38	32.16 $\pm$ 16.14	2.13 $\pm$ 0.10	17.83 $\pm$ 17.83	107.37 $\pm$ 128.29	12.50
Exp	0.47 $\pm$ 0.01	1.92 $\pm$ 0.03	28.60 $\pm$ 61.10	1.18 $\pm$ 0.13	2.59 $\pm$ 0.08	11.21 $\pm$ 3.67	9.50
MLP	0.76 $\pm$ 0.05	2.77 $\pm$ 0.35	5.09 $\pm$ 6.91	0.35 $\pm$ 0.15	2.91 $\pm$ 0.51	14.80 $\pm$ 3.26	10.50
LSTM	0.43 $\pm$ 0.02	1.04 $\pm$ 0.09	0.27 $\pm$ 0.03	0.23 $\pm$ 0.11	1.38 $\pm$ 0.07	6.80 $\pm$ 0.22	3.83
CNN-1D	0.48 $\pm$ 0.05	1.01 $\pm$ 0.06	1.31 $\pm$ 0.60	0.60 $\pm$ 0.75	2.54 $\pm$ 0.48	14.34 $\pm$ 8.16	8.33
TiDE	0.47 $\pm$ 0.04	1.09 $\pm$ 0.04	0.22 $\pm$ 0.01	0.28 $\pm$ 0.10	1.34 $\pm$ 0.21	9.68 $\pm$ 1.58	5.00
TST	0.46 $\pm$ 0.02	0.91 $\pm$ 0.05	0.71 $\pm$ 0.05	0.29 $\pm$ 0.11	1.47 $\pm$ 0.20	16.52 $\pm$ 6.55	6.83
STF	0.41 $\pm$ 0.02	0.99 $\pm$ 0.29	1.92 $\pm$ 1.15	0.26 $\pm$ 0.16	1.92 $\pm$ 0.34	13.79 $\pm$ 4.66	6.17
CF	0.57 $\pm$ 0.06	1.83 $\pm$ 0.18	0.24 $\pm$ 0.02	0.26 $\pm$ 0.10	0.82 $\pm$ 0.34	7.50 $\pm$ 1.02	5.00
PTST	3.71 $\pm$ 0.02	6.34 $\pm$ 0.02	0.45 $\pm$ 0.02	3.58 $\pm$ 0.46	2.06 $\pm$ 0.41	9.80 $\pm$ 1.67	10.33
XGBoost	1.02 $\pm$ 0.00	3.55 $\pm$ 0.00	0.33 $\pm$ 0.00	0.13 $\pm$ 0.00	0.61 $\pm$ 0.00	9.04 $\pm$ 0.00	6.17
TabPFN	0.44 $\pm$ 0.01	1.14 $\pm$ 0.01	0.27 $\pm$ 0.00	0.06 $\pm$ 0.00	0.72 $\pm$ 0.03	7.55 $\pm$ 0.22	3.50
TabDPT	0.50 $\pm$ 0.01	0.90 $\pm$ 0.01	0.25 $\pm$ 0.01	0.10 $\pm$ 0.00	0.69 $\pm$ 0.01	8.63 $\pm$ 0.53	3.33
<i>Original engineering units ($\downarrow$)</i>							
Linear	136.91 $\pm$ 4.13	394.76 $\pm$ 38.18	377375.42 $\pm$ 189407.02	2662.38 $\pm$ 123.32	43203.12 $\pm$ 43216.43	1877895.28 $\pm$ 2475205.90	12.50
Exp	47.10 $\pm$ 1.17	191.83 $\pm$ 2.71	335612.34 $\pm$ 716896.98	1482.75 $\pm$ 160.37	6278.27 $\pm$ 188.56	228864.76 $\pm$ 83234.84	9.50
MLP	75.94 $\pm$ 5.37	276.90 $\pm$ 34.56	59680.85 $\pm$ 81102.09	444.59 $\pm$ 194.00	7060.46 $\pm$ 1246.54	287760.52 $\pm$ 74993.08	10.50
LSTM	43.44 $\pm$ 2.21	104.08 $\pm$ 8.97	3208.61 $\pm$ 369.69	293.94 $\pm$ 136.27	3344.16 $\pm$ 173.37	123688.33 $\pm$ 2403.85	3.83
CNN-1D	48.09 $\pm$ 4.82	101.08 $\pm$ 5.83	15328.28 $\pm$ 7016.36	749.70 $\pm$ 938.67	6162.49 $\pm$ 1160.94	254182.17 $\pm$ 130965.65	8.17
TiDE	46.60 $\pm$ 3.56	109.02 $\pm$ 4.13	2633.01 $\pm$ 107.72	347.09 $\pm$ 128.23	3238.60 $\pm$ 518.75	194915.73 $\pm$ 28818.57	5.17
TST	46.15 $\pm$ 2.03	91.22 $\pm$ 4.96	8293.91 $\pm$ 557.49	358.65 $\pm$ 139.75	3565.53 $\pm$ 491.39	331603.13 $\pm$ 135837.29	6.83
STF	41.31 $\pm$ 2.25	99.50 $\pm$ 29.06	22492.01 $\pm$ 13489.49	329.53 $\pm$ 200.78	4641.65 $\pm$ 816.21	275295.87 $\pm$ 99592.97	6.33
CF	56.62 $\pm$ 6.20	182.98 $\pm$ 17.52	2873.02 $\pm$ 197.96	320.71 $\pm$ 120.56	1986.17 $\pm$ 825.15	149462.60 $\pm$ 25698.73	5.00
PTST	370.79 $\pm$ 2.11	634.40 $\pm$ 2.30	5338.39 $\pm$ 225.45	4483.61 $\pm$ 577.87	4999.49 $\pm$ 987.11	182128.52 $\pm$ 38677.98	10.17
XGBoost	102.50 $\pm$ 0.00	355.02 $\pm$ 0.00	3823.81 $\pm$ 0.00	161.62 $\pm$ 0.00	1484.77 $\pm$ 0.00	176032.70 $\pm$ 0.00	6.17
TabPFN	44.17 $\pm$ 0.59	114.50 $\pm$ 1.47	3121.41 $\pm$ 41.35	79.84 $\pm$ 1.56	1744.71 $\pm$ 73.12	158581.64 $\pm$ 4950.91	3.50
TabDPT	50.30 $\pm$ 0.90	90.43 $\pm$ 0.62	2875.42 $\pm$ 67.81	121.85 $\pm$ 2.64	1675.44 $\pm$ 32.22	163786.30 $\pm$ 10020.04	3.33

in the aggregation order (per-unit-then-mean vs. pooled). The corresponding MSE per-unit-mean variants are omitted from the appendix as they preserve the same ranking with no additional insight.

Per-unit aggregation tightens the gap among the top three (TabDPT, TabPFN, LSTM) on NB14 and UNIBO21, where pooled-MAE differences were inflated by long-trajectory units, and reorders the middle of the table on XJTU-SY. The headline that tabular foundation models lead on average rank is unchanged.

C.6.3. Domain-specific prognostic scores

Two community-standard scores are reported on the families they apply to. The *NASA score* is defined for direct-RUL targets and is reported on the N-CMAPSS families (NC-DS02, NC-P) and PHME20; it asymmetrically penalizes late predictions. The *PHM score* is the bearing/battery-prognostics convention and is reported on NB14, UNIBO21, and XJTU-SY. Per-task scoping is enforced by the framework’s metric registry (Section 4.3); the two scores are presented together in Table 16 as two blocks of one table because they apply to disjoint family sets.

Domain-specific scores mostly track MAE/MSE, but they surface two important failure modes. On the NASA score, a small number of catastrophically late predictions on PHME20 causes MLP and PTST to degrade by orders of magnitude under the asymmetric penalty—a pattern that MAE/MSE can hide. On the PHM score, CF achieves the best score on XJTU-SY despite being mid-tier on MAE, reflecting the score’s emphasis on end-of-life behavior. Reporting symmetric errors alongside domain scores therefore highlights behaviors that any single metric would miss.

C.7. Data-Efficiency Scaling

This section supports the main-paper claim that data efficiency depends not only on the number of available training windows, but also on whether the context covers the relevant operating regimes, degradation stages, and diagnostic classes. The additional scaling figure compares aggregate and blockwise subsampling for PHME20, Unibo, and MZVAV. It extends the main-results discussion by showing when a small context remains representative and when blockwise subsampling removes important trajectory segments or class coverage.

Table 15

Per-unit-mean MAE on battery and bearing prognostics. Top block: normalized target space ($\times 100$) ($\downarrow$); bottom block: original engineering units ($\downarrow$). Computed by the per_unit evaluator: one error per monitored unit, then averaged across units.

Model	NB14	Unibo	XJTU-SY	Average rank
<i>Normalized target space ($\downarrow$)</i>				
Linear	38.58 $\pm$ 9.96	31.66 $\pm$ 21.46	75.37 $\pm$ 47.26	13.00
Exp	30.74 $\pm$ 48.55	10.93 $\pm$ 0.18	21.38 $\pm$ 1.44	10.00
MLP	14.83 $\pm$ 9.66	13.88 $\pm$ 1.78	25.21 $\pm$ 2.73	11.33
LSTM	4.49 $\pm$ 0.32	7.02 $\pm$ 0.71	18.57 $\pm$ 0.50	4.33
CNN-1D	9.12 $\pm$ 1.43	11.74 $\pm$ 1.15	26.13 $\pm$ 6.63	10.67
TiDE	4.02 $\pm$ 0.12	6.30 $\pm$ 0.64	21.28 $\pm$ 2.55	4.33
TST	7.20 $\pm$ 0.26	7.39 $\pm$ 1.18	25.08 $\pm$ 6.06	8.33
STF	10.16 $\pm$ 3.03	8.10 $\pm$ 0.72	21.44 $\pm$ 2.55	9.00
CF	4.14 $\pm$ 0.22	5.43 $\pm$ 1.01	18.87 $\pm$ 0.43	3.67
PTST	5.57 $\pm$ 0.15	10.56 $\pm$ 0.97	21.13 $\pm$ 1.16	7.33
XGBoost	4.82 $\pm$ 0.00	3.59 $\pm$ 0.00	19.20 $\pm$ 0.00	4.33
TabPFN	4.07 $\pm$ 0.02	3.25 $\pm$ 0.07	17.83 $\pm$ 0.13	1.33
TabDPT	4.09 $\pm$ 0.08	3.57 $\pm$ 0.15	20.09 $\pm$ 0.30	3.33
<i>Original engineering units ($\downarrow$)</i>				
Linear	417.94 $\pm$ 107.84	155.85 $\pm$ 105.63	434.14 $\pm$ 341.49	13.00
Exp	333.02 $\pm$ 525.87	53.79 $\pm$ 0.91	153.84 $\pm$ 22.97	10.00
MLP	160.68 $\pm$ 104.69	68.31 $\pm$ 8.74	173.19 $\pm$ 15.11	11.00
LSTM	48.67 $\pm$ 3.42	34.55 $\pm$ 3.51	123.73 $\pm$ 2.27	4.00
CNN-1D	98.76 $\pm$ 15.52	57.78 $\pm$ 5.68	175.36 $\pm$ 46.62	10.33
TiDE	43.52 $\pm$ 1.29	31.02 $\pm$ 3.17	141.91 $\pm$ 13.43	4.00
TST	77.96 $\pm$ 2.77	36.37 $\pm$ 5.79	188.24 $\pm$ 43.65	9.00
STF	110.11 $\pm$ 32.79	39.87 $\pm$ 3.52	161.01 $\pm$ 22.65	9.00
CF	44.84 $\pm$ 2.43	26.75 $\pm$ 4.98	124.85 $\pm$ 6.01	3.33
PTST	60.29 $\pm$ 1.65	51.99 $\pm$ 4.78	143.67 $\pm$ 8.34	7.67
XGBoost	52.24 $\pm$ 0.00	17.68 $\pm$ 0.00	138.98 $\pm$ 0.00	4.67
TabPFN	44.10 $\pm$ 0.25	16.00 $\pm$ 0.32	125.89 $\pm$ 1.97	2.00
TabDPT	44.26 $\pm$ 0.83	17.58 $\pm$ 0.72	131.36 $\pm$ 2.54	3.00

Table 16

Domain-specific prognostic scores. Top block: NASA score on direct-RUL families (NC-DS02, NC-P, PHME20; ↓); bottom block: PHM score (×100) on battery and bearing prognostics (NB14, UNIBO21, XJTU-SY; ↑). Per-task scoping is enforced by the framework’s metric registry. Note that the two scores apply to disjoint family sets and use opposite directions.

Model	NC-DS02	NC-P	PHME20	Average rank
<i>NASA score on direct-RUL families (↓)</i>				
Linear	2.03 ± 0.07	7.90 ± 2.31	3044.09 ± 2526.30	11.00
Exp	0.85 ± 0.02	2.69 ± 0.04	229.49 ± 39.40	6.67
MLP	2.53 ± 1.62	4910.51 ± 10876.24	481046.65 ± 1027047.25	12.33
LSTM	0.81 ± 0.04	1.46 ± 0.13	8.50 ± 5.84	4.00
CNN-1D	0.87 ± 0.09	1.44 ± 0.09	2709.03 ± 5992.51	7.00
TiDE	0.86 ± 0.06	1.69 ± 0.06	11.82 ± 9.43	6.00
TST	0.85 ± 0.03	1.26 ± 0.09	12.64 ± 7.24	4.00
STF	0.78 ± 0.03	1.30 ± 0.33	21.05 ± 30.49	3.33
CF	1.01 ± 0.11	2.99 ± 0.18	2215.75 ± 4653.64	9.00
PTST	6.11 ± 0.12	10.69 ± 0.04	19528004.00 ± 11187183.05	12.67
XGBoost	1.62 ± 0.00	6.10 ± 0.00	3.95 ± 0.00	7.33
TabPFN	0.80 ± 0.01	1.49 ± 0.03	1.21 ± 0.02	3.00
TabDPT	0.91 ± 0.02	1.31 ± 0.01	5.94 ± 2.04	4.67
Model	NB14	Unibo	XJTU-SY	Average rank
<i>PHM score on battery and bearing prognostics (↑)</i>				
Linear	5.38 ± 1.51	5.85 ± 2.55	11.40 ± 10.24	13.00
Exp	14.45 ± 8.13	9.91 ± 0.30	19.15 ± 4.44	10.67
MLP	15.46 ± 6.30	10.85 ± 1.11	18.27 ± 2.20	10.00
LSTM	31.28 ± 2.21	14.74 ± 0.46	24.83 ± 0.78	3.00
CNN-1D	18.29 ± 2.08	10.70 ± 1.32	18.36 ± 4.77	10.00
TiDE	36.28 ± 1.72	14.63 ± 2.39	22.46 ± 2.77	4.33
TST	24.31 ± 0.92	13.71 ± 1.41	14.89 ± 7.81	8.67
STF	14.34 ± 6.88	10.79 ± 1.27	21.58 ± 3.29	9.67
CF	33.45 ± 1.31	17.21 ± 1.78	24.72 ± 1.78	2.67
PTST	24.17 ± 0.71	12.57 ± 1.66	23.41 ± 1.61	7.00
XGBoost	24.49 ± 0.00	20.24 ± 0.00	20.14 ± 0.00	5.67
TabPFN	26.03 ± 0.28	21.59 ± 0.41	24.40 ± 0.63	3.33
TabDPT	29.16 ± 0.40	20.35 ± 0.73	24.42 ± 1.01	3.00

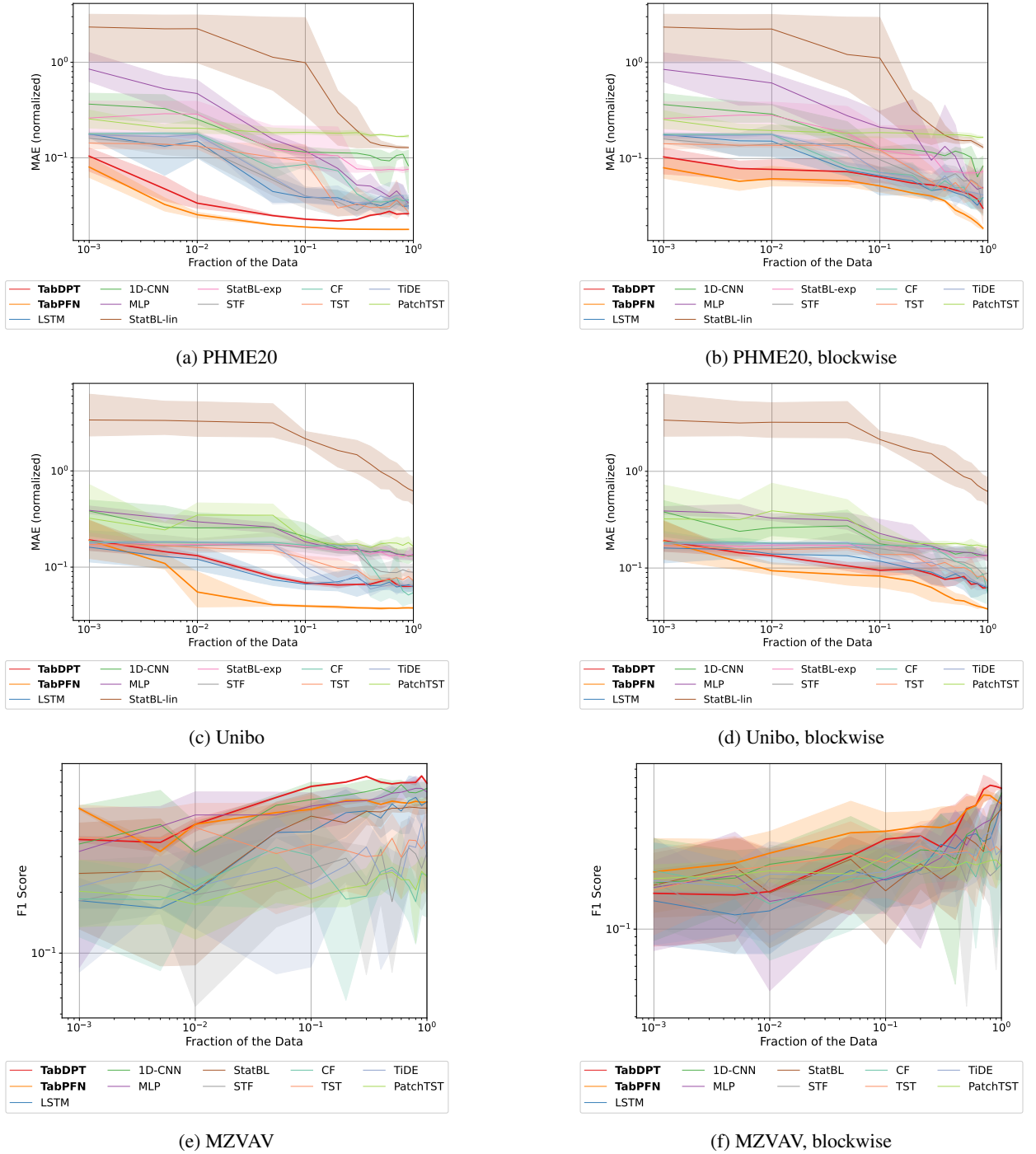


Figure 8: Data-efficiency scaling under aggregate and blockwise context subsampling for PHME20, Unibo, and MZVAV. Each row compares aggregate random subsampling with contiguous blockwise subsampling, highlighting when small contexts preserve or lose coverage of trajectories or diagnostic classes.

D. Reproducibility

D.1. Framework-Based Execution

All reported experiments are executed through the PICID evaluation framework [52]. In this paper, reproducibility therefore means that the benchmark choices described in the method and dataset appendices are fixed as executable experiment configurations rather than reconstructed manually from prose.

D.2. Configuration-Fixed Experiments

Each experiment is wrapped in a resolved configuration that fixes the datasource, split, transform pipeline, target construction, sequence-slicing parameters, tabularization rule, model family, hyperparameters, evaluator, seed, and validation-selection rule. These configuration choices define the experiment from datasource construction to metric computation, so a reported number is tied to a concrete protocol instance.

The run record should identify:

- the resolved configuration, including command-line overrides;
- the code revision and dependency environment;
- the dataset source or local path configuration used for execution;
- the saved evaluation summaries, predictions, and plots used in the manuscript.

D.3. Shared Preprocessing and Evaluation Boundaries

The same configured preprocessing and evaluation boundaries are used across the model families compared in the paper. Dataset splits are reported in the dataset-protocol section above, model selection uses validation data, and final metrics are computed on the held-out test partition.

The leakage constraints are those defined in the formal protocol. Parameters fitted by feature transformations $\mathcal{G}$, target transformations $\mathcal{H}$, target-alignment choices inside $\tilde{\mathcal{H}}$, normalization statistics, and tabularization choices are determined only from the training partition. Once fitted, these choices are frozen and applied unchanged to validation and test partitions.

For tabular in-context models, labeled context examples are drawn only from the training partition. For intra-unit temporal splits, context selection must also respect temporal order so that no future-derived sample is used to predict an earlier query. Machine-specific paths are kept separate from the experimental protocol through path configurations, so local storage layout does not change the scientific comparison.

E. Data and Code Access

The public repository link is recorded as github.com/EPFL-IMOS/picid. An archival artifact identifier can be added before submission if one is created.

E.1. Code and Protocol Availability

The implementation is distributed as a Python project with version-controlled experiment configurations. These configurations define the datasource, preprocessing pipeline, task definition, model, and evaluator used for each reported benchmark setting, while machine-specific dataset and artifact locations are supplied separately through path configurations.

The code repository and experiment configurations are linked as github.com/EPFL-IMOS/picid.

E.2. Third-Party Datasets

The evaluated datasets are third-party benchmark datasets and should be obtained according to their respective source terms rather than redistributed as part of the paper. The evaluated datasets are NB14 [5], UNIBO21 [57], XJTU-SY [61], N-CMAPSS [4, 20], HSF15 [25], PHME20 [29], and MZVAV [21].

The implementation supports local dataset paths through the selected path configuration. Dataset access therefore remains separate from the benchmark protocol: users obtain the third-party data from the appropriate source, place or cache them locally, and then run the same configured preprocessing and evaluation pipeline.

References

- [1] Ansari, A.F., Shchur, O., Küken, J., Auer, A., Han, B., Mercado, P., Rangapuram, S.S., Shen, H., Stella, L., Zhang, X., Goswami, M., Kapoor, S., Maddix, D.C., Guérroun, P., Hu, T., Yin, J., Erickson, N., Desai, P.M., Wang, H., Rangwala, H., Karypis, G., Wang, Y., Bohlke-Schneider, M., 2025. Chronos-2: From univariate to universal forecasting. URL: <https://arxiv.org/abs/2510.15821>, arXiv:2510.15821.
- [2] Ansari, A.F., Stella, L., Turkmen, A.C., Zhang, X., Mercado, P., Shen, H., Shchur, O., Rangapuram, S.S., Arango, S.P., Kapoor, S., Zschiegner, J., Maddix, D.C., Wang, H., Mahoney, M.W., Torkkola, K., Wilson, A.G., Bohlke-Schneider, M., Wang, B., 2024. Chronos: Learning the language of time series. URL: <https://openreview.net/forum?id=gerNCVqqtR>. expert Certification.
- [3] Arbel, M., Salinas, D., Hutter, F., 2026. Equitabpf: A target-permutation equivariant prior fitted network, in: Advances in Neural Information Processing Systems, Curran Associates, Inc., pp. 62586–62609. URL: https://proceedings.neurips.cc/paper_files/paper/2025/file/5a66c7adffdbde9dd5e78820cbf6935c-Paper-Conference.pdf.
- [4] Arias Chao, M., Kulkarni, C., Goebel, K., Fink, O., 2021. Aircraft engine run-to-failure dataset under real flight conditions for prognostics and diagnostics. Data 6, 5. doi:10.3390/data6010005.
- [5] Bole, B., Kulkarni, C.S., Daigle, M., 2014. Adaptation of an electrochemistry-based li-ion battery model to account for deterioration observed under randomized use. Annual Conference of the PHM Society 6. doi:10.36001/phmconf.2014.v6i1.2490.
- [6] Bosello, M., Falcomer, C., Rossi, C., Pau, G., 2023. To charge or to sell? ev pack useful life estimation via lstms, cnns, and autoencoders. Energies 16, 2837. doi:10.3390/en16062837.
- [7] Cai, S., Sun, X., Zhong, H., 2025. Explore the time series forecasting potential of tabpf: leveraging the intrinsic periodicity of data, in: ICML 2025 Workshop on Foundation Models for Structured Data (FMSD). URL: <https://openreview.net/forum?id=7JGD1kNlZU>. workshop paper.
- [8] Case Western Reserve University Bearing Data Center, . Case Western Reserve University Bearing Data Center Website. <https://engineering.case.edu/bearingdatacenter/download-data-file>. Accessed: 2026-05-29.
- [9] Cha, M., Yoon, S.i., Kim, S., Kang, D., Nam, K., Lee, T., Kim, J.Y., 2025. Large language model-based autonomous agent for prognostics and health management. Machines 13, 831. URL: <https://www.mdpi.com/2075-1702/13/9/831>, doi:10.3390/machines13090831.
- [10] Chen, T., Guestrin, C., 2016. Xgboost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794. doi:10.1145/2939672.2939785.
- [11] Das, A., Kong, W., Leach, A., Mathur, S.K., Sen, R., Yu, R., 2023. Long-term forecasting with tide: Time-series dense encoder. Transactions on Machine Learning Research .
- [12] Das, A., Kong, W., Sen, R., Zhou, Y., 2024. A decoder-only foundation model for time-series forecasting, in: Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., Berkenkamp, F. (Eds.), Proceedings of the 41st International Conference on Machine Learning, PMLR. pp. 10148–10167. URL: <https://proceedings.mlr.press/v235/das24c.html>.
- [13] Dong, Q., Li, L., Dai, D., Zheng, C., Ma, J., Li, R., Xia, H., Xu, J., Wu, Z., Chang, B., et al., 2024. A survey on in-context learning, in: Proceedings of the 2024 conference on empirical methods in natural language processing, pp. 1107–1128.
- [14] Dooley, S., Khurana, G.S., Mohapatra, C., Naidu, S.V., White, C., 2024. Forecastpf: Synthetically-trained zero-shot forecasting. Advances in Neural Information Processing Systems 36.
- [15] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.E., Lomeli, M., Hosseini, L., Jégou, H., 2024. The faiss library. URL: <https://arxiv.org/abs/2401.08281>, doi:10.48550/arXiv.2401.08281, arXiv:2401.08281.
- [16] Eldele, E., Ragab, M., Qing, X., Chen, Z., Wu, M., Li, X., Lee, J., et al., 2025. Unifault: A fault diagnosis foundation model from bearing data. arXiv preprint arXiv:2504.01373 .
- [17] Feng, C., Huang, L., Krompass, D., . Only the curve shape matters: Training foundation models for zero-shot multivariate time series forecasting through next curve shape prediction. URL: <http://arxiv.org/abs/2402.07570>, arXiv:2402.07570 [cs].
- [18] Feng, T., Chen, Y., Tsai, C.Y., Sun, Y., Das, A., El Maghraoui, K., Lin, S., Patel, D., 2026. PHMForge: Evaluating llm agents on industrial prognostics through mcp-native, algorithm-grounded tools. URL: <https://arxiv.org/abs/2604.01532>, doi:10.48550/arXiv.2604.01532, arXiv:2604.01532.
- [19] Fink, O., Nejjar, I., Sharma, V., Faghih Niresi, K., Sun, H., Dong, H., Xu, C., Wei, A., Bizzi, A., Theiler, R., Tian, Y., Von Krannichfeldt, L., Ma, Z., Garmaev, S., Zhang, Z., Zhao, M., Steiner, K., Kesmen, Y., 2026. From physics to machine learning and back: Part II - Learning and observational bias in prognostics and health management (PHM). Reliability Engineering & System Safety 274, 112376. doi:10.1016/j.res.2026.112376.
- [20] Frederick, D.K., DeCastro, J.A., Litt, J.S., 2007. User's guide for the commercial modular aero-propulsion system simulation (C-MAPSS). Technical Report NASA/TM—2007-215026. NASA Glenn Research Center.
- [21] Granderson, J., Lin, G., Harding, A., Im, P., Chen, Y., 2020a. Building fault detection data to aid diagnostic algorithm creation and performance testing. Scientific Data 7, 65. URL: <https://www.nature.com/articles/s41597-020-0398-6>, doi:10.1038/s41597-020-0398-6.
- [22] Granderson, J., Lin, G., Harding, A., Im, P., Chen, Y., 2020b. Dataset for building fault detection and diagnostics algorithm creation and performance testing URL: https://figshare.com/articles/dataset/LBNLDataSynthesisInventory_pdf/11752740, doi:10.6084/m9.figshare.11752740.v3.
- [23] Grigsby, J., Wang, Z., Qi, Y., 2021. Long-range transformers for dynamic spatiotemporal forecasting. arXiv:2109.12218.
- [24] Hagmeyer, S., Mauthe, F., Zeiler, P., 2021. Creation of publicly available data sets for prognostics and diagnostics addressing data scenarios relevant to industrial applications. International Journal of Prognostics and Health Management 12.
- [25] Helwig, N., Pignatelli, E., Schütze, A., 2015. Condition monitoring of a complex hydraulic system using multivariate statistics, in: 2015 IEEE International Instrumentation and Measurement Technology Conference (I2MTC) Proceedings, pp. 210–215. doi:10.1109/I2MTC.2015.7151267.
- [26] Hollmann, N., Müller, S., Eggenberger, K., Hutter, F., 2023. Tabpf: A transformer that solves small tabular classification problems in a second, in: The Eleventh International Conference on Learning Representations.

- [27] Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S.B., Schirrmeister, R.T., Hutter, F., 2025. Accurate predictions on small data with a tabular foundation model. *Nature* 637, 319–326.
- [28] Hoo, S.B., Müller, S., Salinas, D., Hutter, F., 2024. The tabular foundation model tabpfm outperforms specialized time series forecasting models based on simple features, in: *NeurIPS 2024 Workshop on Table Representation Learning (TRL)*. URL: <https://neurips.cc/virtual/2024/103164>. neurIPS 2024 workshop paper.
- [29] İnce, K., Sirkeci, E., Genç, Y., 2020. Remaining useful life prediction for experimental filtration system: A data challenge. *PHM Society European Conference* 5. doi:10.36001/phme.2020.v5i1.1317.
- [30] Jin, M., Wang, S., Ma, L., Chu, Z., Zhang, J., Shi, X., Chen, P.Y., Liang, Y., Li, Y.F., Pan, S., Wen, Q., 2024. Time-llm: Time series forecasting by reprogramming large language models, in: *International Conference on Learning Representations*, pp. 23857–23880. URL: https://proceedings.iclr.cc/paper_files/paper/2024/file/680b2a8135b9c71278a09cafb605869e-Paper-Conference.pdf.
- [31] Kim, M.J., Grinsztajn, L., Varoquaux, G., 2024. Carte: Pretraining and transfer for tabular learning, in: *Forty-first International Conference on Machine Learning*.
- [32] LeCun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. *Nature* 521, 436–444. doi:10.1038/nature14539.
- [33] Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C.H., Kang, J., 2019. Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics* 36, 1234–1240. URL: <http://dx.doi.org/10.1093/bioinformatics/btz682>, doi:10.1093/bioinformatics/btz682.
- [34] Lei, Y., Li, N., Guo, L., Li, N., Yan, T., Lin, J., 2018. Machinery health prognostics: A systematic review from data acquisition to remaining useful life prediction. *Mechanical Systems and Signal Processing* 104, 799–834. doi:10.1016/j.ymssp.2017.11.016.
- [35] Li, C., Li, S., Feng, Y., Gryllias, K., Gu, F., Pecht, M., 2024. Small data challenges for intelligent prognostics and health management: a review. *Artificial Intelligence Review* 57, 214.
- [36] Liu, D., Qu, G., Xu, Y., Qiu, T., Ding, S., Guo, K., 2025. ICL4RUL: In-context learning-based aircraft engine remaining useful life prediction. *IEEE Internet of Things Journal* 12, 29766–29783. URL: <https://ieeexplore.ieee.org/document/10998956>, doi:10.1109/JIOT.2025.3569131.
- [37] Liu, Y., Zhang, H., Li, C., Huang, X., Wang, J., Long, M., . Timer: Generative pre-trained transformers are large time series models. URL: <http://arxiv.org/abs/2402.02368>, arXiv:2402.02368 [cs, stat].
- [38] Ma, J., Thomas, V., Hosseinzadeh, R., Labach, A., Kamkari, H., Cresswell, J.C., Golestan, K., Yu, G., Caterini, A.L., Volkovs, M., 2025. TabDPT: Scaling tabular foundation models on real data URL: <https://openreview.net/forum?id=pIZxE0ZCId>.
- [39] Magadán, L., Roldán-Gómez, J., Granda, J., Suárez, F., 2023. Early fault classification in rotating machinery with limited data using tabpfm. *IEEE Sensors Journal* 23, 30960–30970. doi:10.1109/JSEN.2023.3331100.
- [40] Mauthe, F., Steinmann, L., Neu, M., Zeiler, P., 2025. Overview and analysis of publicly available degradation data sets for tasks within prognostics and health management, in: *35th European safety and reliability conference*.(accepted). Research Publishing.
- [41] Miller, J.A., Aldosari, M., Saeed, F., Barna, N.H., Rana, S., Arpinar, I.B., Liu, N., . A survey of deep learning and foundation models for time series forecasting. URL: <http://arxiv.org/abs/2401.13912>, doi:10.48550/arXiv.2401.13912, arXiv:2401.13912 [cs].
- [42] Nie, Y., H. Nguyen, N., Sinthong, P., Kalagnanam, J., 2023. A time series is worth 64 words: Long-term forecasting with transformers, in: *International Conference on Learning Representations*.
- [43] Nori, H., Lee, Y.T., Zhang, S., Carignan, D., Edgar, R., Fusi, N., King, N., Larson, J., Li, Y., Liu, W., Luo, R., McKinney, S.M., Ness, R.O., Poon, H., Qin, T., Usuyama, N., White, C., Horvitz, E., 2023. Can generalist foundation models outcompete special-purpose tuning? case study in medicine. arXiv preprint arXiv:2311.16452 URL: <https://arxiv.org/abs/2311.16452>.
- [44] Qiao, X., Liow, H.Y., Jauw, V.L., Lim, C.S., 2025. A comparative study of deep learning model based equipment fault diagnosis and prognosis. *International Journal of Prognostics and Health Management* 16. doi:10.36001/IJPHM.2025.v16i1.4254.
- [45] Qu, J., Holzmüller, D., Varoquaux, G., Morvan, M.L., 2025. Tabicl: A tabular foundation model for in-context learning on large data, in: *Proceedings of the 38th International Conference on Machine Learning (ICML 2025)*. URL: <https://arxiv.org/abs/2502.05564>. accepted at ICML 2025.
- [46] Ramasso, E., Saxena, A., 2014. Performance Benchmarking and Analysis of Prognostic Methods for CMAPSS Datasets. *International Journal of Prognostics and Health Management* 5. URL: <https://papers.phmsociety.org/index.php/ijphm/article/view/2236>, doi:10.36001/ijphm.2014.v5i2.2236.
- [47] Salinas-Camus, M., Goebel, K., Eleftheroglou, N., 2025. A comprehensive review and evaluation framework for data-driven prognostics: Uncertainty, robustness, interpretability, and feasibility. *Mechanical Systems and Signal Processing* 237, 113015. doi:10.1016/j.ymssp.2025.113015.
- [48] Saxena, A., Goebel, K., Simon, D., Eklund, N., 2008. Damage propagation modeling for aircraft engine run-to-failure simulation, in: *2008 International Conference on Prognostics and Health Management*, pp. 1–9. doi:10.1109/PHM.2008.4711414.
- [49] Siarni-Namini, S., Tavakoli, N., Siarni Namin, A., 2019. The performance of lstm and bilstm in forecasting time series, in: *2019 IEEE International Conference on Big Data (Big Data)*, pp. 3285–3292. doi:10.1109/BigData47090.2019.9005997.
- [50] Sim, J., Kim, S., Park, H.J., Choi, J.H., 2020. A tutorial for feature engineering in the prognostics and health management of gears and bearings. *Applied Sciences* 10, 5639. doi:10.3390/app10165639.
- [51] Sun, L., Wu, J., Wang, J., Wen, S., Li, G., Liu, Y., 2025. Fault diagnosis of slewing bearing using audible sound signal based on time gan–tabpfm method. *Journal of Vibration and Acoustics* 147, 041002. doi:10.1115/1.4068223.
- [52] Telyatnikov, L., Theiler, R., Von Krannichfeldt, L., Fink, O., 2026. Picid: A modular evaluation infrastructure for reproducible phm across tasks and domains. URL: <https://arxiv.org/abs/2605.28345>, doi:10.48550/arXiv.2605.28345, arXiv:2605.28345.
- [53] Theiler, R., Comito, L., Leko, D., Von Krannichfeldt, L., Telyatnikov, L., Fink, O., 2026. From paper to benchmark: agentic, framework-based reproduction of under-specified methods in machine health intelligence. URL: <https://arxiv.org/abs/2605.28371>, doi:10.48550/arXiv.2605.28371, arXiv:2605.28371.

- [54] Tu, S., Zhang, Y., Zhang, J., Fu, Z., Zhang, Y., Yang, Y., 2024. Powerpm: Foundation model for power systems, in: Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., Zhang, C. (Eds.), *Advances in Neural Information Processing Systems*, Curran Associates, Inc.. pp. 115233–115260. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/d0a2279c9f7ded859bcbf878c3c3d1ed-Paper-Conference.pdf, doi:10.52202/079017-3659.
- [55] Wang, J., Ma, Y., Zhang, L., Gao, R.X., Wu, D., 2018. Deep learning for smart manufacturing: Methods and applications. *Journal of Manufacturing Systems* 48, 144–156. doi:10.1016/j.jmsy.2018.01.003.
- [56] Wang, Y., Wu, H., Dong, J., Liu, Y., Wang, C., Long, M., Wang, J., 2026. Deep time series models: A comprehensive survey and benchmark. *IEEE Transactions on Pattern Analysis and Machine Intelligence* URL: <https://doi.org/10.1109/TPAMI.2026.3690845>, doi:10.1109/TPAMI.2026.3690845.
- [57] Wong, K.L., Bosello, M., Tse, R., Falcomer, C., Rossi, C., Pau, G., 2021a. Li-ion batteries state-of-charge estimation using deep lstm at various battery specifications and discharge cycles, in: *Proceedings of the Conference on Information Technology for Social Good*, Association for Computing Machinery, New York, NY, USA. p. 85–90. URL: <https://doi.org/10.1145/3462203.3475878>, doi:10.1145/3462203.3475878.
- [58] Wong, K.L., Bosello, M., Tse, R., Falcomer, C., Rossi, C., Pau, G., 2021b. Li-ion batteries state-of-charge estimation using deep lstm at various battery specifications and discharge cycles, in: *Proceedings of the Conference on Information Technology for Social Good*, Association for Computing Machinery, New York, NY, USA. p. 85–90. URL: <https://doi.org/10.1145/3462203.3475878>, doi:10.1145/3462203.3475878.
- [59] Woo, G., Liu, C., Kumar, A., Xiong, C., Savarese, S., Sahoo, D., 2024. Unified training of universal time series forecasting transformers, in: Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., Berkenkamp, F. (Eds.), *Proceedings of the 41st International Conference on Machine Learning*, PMLR. pp. 53140–53164. URL: <https://proceedings.mlr.press/v235/woo24a.html>.
- [60] Yadan, O., 2019. Hydra - a framework for elegantly configuring complex applications. Github. URL: <https://github.com/facebookresearch/hydra>.
- [61] Yaguo, L., Tianyu, H., Biao, W., Naipeng, L., Tao, Y., Jun, Y., 2019. Xjtu-sy rolling element bearing accelerated life test datasets: A tutorial. *Journal of Mechanical Engineering* 55, 1–6.
- [62] Yao, J., Han, T., 2026. Utilizing large-scale foundation models for prognostics and health management in wind turbines: Techniques, challenges, and future directions. *Renewable and Sustainable Energy Reviews* 227, 116527. doi:10.1016/j.rser.2025.116527.
- [63] Zhang, S., Wang, T., Kulkarni, A., Adams, S., Bhattacharya, S., Tiyyagura, S.R., Bowen, E., Veeramani, B., Zhou, D., 2025. PDMBench: A Standardized Platform for Predictive Maintenance Research. URL: <https://openreview.net/forum?id=oJhJ8w0CNB>. openReview preprint.
- [64] Zhang, Y., Yan, J., 2023. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: *The eleventh international conference on learning representations*.
- [65] Zhao, R., Yan, R., Chen, Z., Mao, K., Wang, P., Gao, R.X., 2019. Deep learning and its applications to machine health monitoring. *Mechanical Systems and Signal Processing* 115, 213–237. doi:10.1016/j.ymssp.2018.05.050.
- [66] Zhou, C., Li, Q., Li, C., Yu, J., Liu, Y., Wang, G., Zhang, K., Ji, C., Yan, Q., He, L., et al., 2025. A comprehensive survey on pretrained foundation models: A history from bert to chatgpt. *International Journal of Machine Learning and Cybernetics* 16, 9851–9915.
- [67] Zonta, T., da Costa, C.A., da Rosa Righi, R., de Lima, M.J., da Trindade, E.S., Li, G., 2020. Predictive maintenance in the industry 4.0: A systematic literature review. *Computers & Industrial Engineering* 150, 106889. doi:10.1016/j.cie.2020.106889.